

Local Feature Filtering for Scalable and Well-Conditioned Domain-Decomposed Random Feature Methods

Jan Willem van Beek^a, Victorita Dolean^{a,*}, Ben Moseley^b

^a*Eindhoven University of Technology, PO Box 513, Eindhoven, 5600 MB, The Netherlands*

^b*Imperial College London, Exhibition Rd, South Kensington, London, SW7 2AZ, United Kingdom*

Abstract

Random Feature Methods (RFMs) [1] and their variants such as extreme learning machine finite-basis physics-informed neural networks (ELM-FBPINNs) [2] offer a scalable approach for solving partial differential equations (PDEs) by using localized, overlapping and randomly initialized neural network basis functions to approximate the PDE solution and training them to minimize PDE residuals through solving structured least-squares problems. This combination leverages the approximation power of randomized neural networks, the parallelism of domain decomposition, and the accuracy and efficiency of least-squares solvers. However, the resulting structured least-squares systems are often *severely ill-conditioned*, due to local redundancy among random basis functions and correlation introduced by subdomain overlaps, which significantly affects the convergence of standard solvers.

In this work, we introduce a *block rank-revealing QR (RRQR) filtering and preconditioning* strategy that operates directly on the structured least-squares problem. First, *local RRQR factorizations identify and remove redundant basis functions while preserving numerically informative ones, reducing problem size, and improving conditioning*. Second, we use these factorizations to construct a right preconditioner for the global problem which preserves *block-sparsity and numerical stability*. Third, we derive deterministic bounds of the condition number of the preconditioned system, with probabilistic refinements for small overlaps.

We validate our approach on challenging, multi-scale PDE problems in 1D, 2D, and (2 + 1)D, demonstrating reductions in condition numbers by up to eleven orders of magnitude, LSQR convergence speedups by factors of 10–1000, and higher accuracy than both unpreconditioned and additive Schwarz-preconditioned baselines, all at significantly lower memory and computational cost. These results establish RRQR-based preconditioning as a scalable, accurate, and efficient enhancement for RFM-based PDE solvers.

Keywords: random feature methods, physics-informed machine learning, overlapping domain decomposition, rank-revealing QR factorization, preconditioning of least-squares problems, numerical solution of PDEs
2000 MSC: 65N55, 65N35, 65F10

1. Introduction

Accurate and efficient numerical solution of partial differential equations (PDEs) is a fundamental challenge in computational science and engineering. Traditional discretization approaches such as finite difference (FDM) and finite element methods (FEM) are well-developed and robust but can face difficulties in high-dimensional, complex-geometry, or multi-scale settings, where accuracy, efficiency, and scalability can be hard to balance.

In recent years, *scientific machine learning* (SciML) has emerged as a promising alternative. Among SciML methods, *physics-informed neural networks* (PINNs) [3, 4] recast the PDE solution into a residual minimization problem, using a neural network to directly approximate the solution and enabling a mesh-free solution and easy integration with inverse problems. The idea of neural-network-based PDE solvers dates back to [5], and the

*Corresponding author

Email addresses: j.w.v.beek@tue.nl (Jan Willem van Beek), v.dolean.maini@tue.nl (Victorita Dolean), b.moseley@imperial.ac.uk (Ben Moseley)

approach was extended by [3] to handle boundary and initial value problems by constructing trial functions that satisfy boundary conditions exactly.

While PINNs offer appealing properties, they exhibit well-known limitations: they suffer from spectral bias [6], struggle with multi-scale PDEs [7, 8], and exhibit super-linear growth in training complexity [9, 7]. Domain decomposition strategies have been proposed to address these shortcomings. Conservative PINNs (cPINNs) [10] enforce flux continuity across non-overlapping subdomains; extended PINNs (XPINNs) [11] generalize to arbitrary space-time decompositions; *hp*-variational PINNs (*hp*-VPINNs) [12] incorporate piecewise polynomial test functions; and the deep domain decomposition method (DeepDDM) [13] applies Schwarz iterations. Finite-basis PINNs (FBPINNs) [7] combine overlapping Schwarz decomposition with localized neural networks, later enhanced by iterative training techniques [14] and extended to multilevel decomposition [15].

In FBPINNs, the solution domain Ω is split into overlapping subdomains $\Omega = \cup_{j=1}^J \Omega_j$, each equipped with a neural network $u_j(\mathbf{x}, \theta_j)$, with $\mathbf{x}$ denoting coordinates in the domain and θ_j denoting the network’s learnable parameters. The global solution approximation is constructed as $u(\mathbf{x}, \theta) = \sum_{j=1}^J \omega_j(\mathbf{x}) u_j(\mathbf{x}, \theta_j)$, where $\omega_j(\mathbf{x})$ are smooth, localized window functions forming a partition of unity. This localization improves accuracy and mitigates spectral bias but leaves the method computationally expensive, since all network parameters must be optimized via non-convex gradient descent.

A powerful acceleration idea is to replace subdomain networks with *extreme learning machines* (ELMs) [16], where hidden layer weights are randomized and fixed, which reduces training to solving a linear least-squares problem for the output weights. In this way, the PDE solution is approximated with localized, overlapping and randomly initialized neural network basis functions (or *features*) and yields a domain-decomposed variant of the *Random Feature Method* (RFM) [1], which is akin to earlier randomized network paradigms such as random vector functional-links (RVFLs) [17] and reservoir computing [18]. RFMs have also been combined with structured sampling strategies, such as the SWIM method [19], to improve accuracy and efficiency.

Despite these advantages, RFMs and ELM-FBPINNs face a major challenge: **the least-squares system is often severely ill-conditioned**. Two main factors seem to contribute in a decisive manner: (i) **Local redundancy**: Within each subdomain, randomly generated features may be nearly linearly dependent; (ii) **Overlap interference**: Features overlapping between neighboring subdomains introduce correlations that affect conditioning.

Recent work by [20] proposed addressing these issues via subdomain singular value decomposition (SVD) for local dimensionality reduction, combined with an additive Schwarz (AS) preconditioner applied to the normal equations of the least-squares system. While this approach can improve solver convergence, it has three significant drawbacks: (i) Preconditioning the normal equations squares the condition number $\kappa(\mathbf{M})$ of the global least-squares matrix, $\mathbf{M}$, (ii) Forming $\mathbf{M}^T \mathbf{M}$ destroys sparsity, and (iii) SVD and AS preconditioner assembly incur high memory and computational cost. In this work, we propose a *direct least-squares preconditioning strategy* based on *subdomain rank-revealing QR (RRQR)* factorizations [21]. The proposed method has three key advantages:

1. **Feature selection**: Local RRQR identifies and retains numerically informative features while discarding redundant ones, reducing problem size and improving conditioning.
2. **Sparse, stable preconditioning**: We assemble a block-structured right preconditioner for the global least-squares problem directly from the filtered QR factors, preserving sparsity and avoiding the stability loss of normal-equation preconditioning.
3. **Theoretical insight**: We provide a deterministic analysis of the condition number for the preconditioned system. Probabilistic refinements, based on a random matrix model for the local feature blocks, are given in Appendix A.

Our contributions are validated through extensive numerical experiments on PDEs in 1D, 2D, and $(2 + 1)$ D. We demonstrate up to eleven orders of magnitude improvements in conditioning, solver convergence speedups by factors of 10–1000, and higher accuracy over both unpreconditioned and AS-preconditioned baselines, at significantly lower memory and computational cost.

The remainder of the paper is organized as follows. In Section 2 we review RFMs, ELM-FBPINNs, and **relevant preconditioning strategies, and recall the RRQR factorization**. In Section 3 we present our proposed RRQR filtering and preconditioning method, together with a theoretical analysis of conditioning in the presence of overlap. In Section 4 we evaluate the method on a range of PDE problems, including strong and weak scaling studies, and

compare against established baselines. We conclude this section with a discussion of limitations and directions for future work.

2. Background

In this section we begin by formulating the Random Feature Method (RFM) for solving boundary value problems (BVPs). The goal is to approximate the solution of a linear PDE using a neural ansatz that combines randomized neural network basis functions (or features) with localized support. By fixing the random feature parameters and optimizing only the output weights, we reduce the problem to a structured least-squares problem. This section details the mathematical setup, introduces the learning task, and reformulates this task into a least-squares problem in matrix form to set the stage for subsequent analysis of conditioning and preconditioning strategies.

2.1. RFM for BVP a.k.a. the ELM-FBPINN method

We start with a PDE defined in a domain $\Omega \subset \mathbb{R}^d$

$$\mathcal{N}[u(\mathbf{x})] = f(\mathbf{x}), \quad \mathbf{x} \in \Omega,$$

under boundary conditions given by

$$\mathcal{B}_l[u(\mathbf{x})] = g_l(\mathbf{x}), \quad \mathbf{x} \in \Gamma_l \subset \partial\Omega, \quad l = 1, \dots, L,$$

where $\mathcal{N}$ and the $\mathcal{B}_l$ are assumed to be linear operators and f and g_l are forcing and boundary functions. We approximate the solution u with a piecewise neural network, $\hat{u} \approx u$,

$$\hat{u}(\mathbf{x}, \mathbf{a}) = \sum_{j=1}^J \omega_j(\mathbf{x}) \sum_{k=1}^K a_{jk} \phi_{jk}(\mathbf{x}, \theta_{jk}), \quad (1)$$

with $\phi_{jk}(\mathbf{x}, \theta_{jk})$ being any parameterised neural network function with a scalar output. In this paper, without loss of generality, we use shallow fully connected networks of the form

$$\begin{aligned} \mathbf{z}_j^{(0)} &= \mathbf{x}, \\ \mathbf{z}_j^{(l)} &= \Xi \left(W_j^{(l)} \mathbf{z}_j^{(l-1)} + b_j^{(l)} \right), \quad l = 1, \dots, h-1, \\ \phi_{jk} &= \Xi \left(\mathbf{w}_{jk}^\top \mathbf{z}_j^{(h-1)} \right), \end{aligned} \quad (2)$$

where Ξ is a nonlinear activation function applied elementwise, and the network parameters are given by $\theta_{jk} = \{W_j^{(l)}, b_j^{(l)}\}_{l=1}^{h-1} \cup \{\mathbf{w}_{jk}\}$. The ω_j are *window functions*, with local support on subdomains Ω_j that cover Ω . These subdomains are allowed to overlap. The a_{jk} are the output weights of each localized neural network.

The parameters θ_{jk} are now randomized and fixed, so only the coefficients a_{jk} remain as free parameters. The idea of randomizing a subset of the parameter space gained prominence in the machine learning space with the publication of [22]. More recently, the combination of feature randomization and locally supported window functions, the latter inspired by domain-decomposition methods, has been explored in [2], [1], [20].

The model parameters $\mathbf{a} = \{a_{jk}\}$ are determined by minimizing a loss functional that penalizes deviations from both the PDE and its boundary conditions at a set of collocation and boundary points, in a similar fashion to PINNs [3, 4]. Specifically, we define the loss function as

$$\mathcal{L}(\mathbf{a}) = \frac{1}{N_I} \sum_{n=1}^{N_I} (\mathcal{N}[\hat{u}(\mathbf{x}_n, \mathbf{a})] - f(\mathbf{x}_n))^2 + \sum_{l=1}^L \frac{\lambda_l}{N_B^{(l)}} \sum_{i=1}^{N_B^{(l)}} (\mathcal{B}_l[\hat{u}(\mathbf{x}_i^{(l)}, \mathbf{a})] - g_l(\mathbf{x}_i^{(l)}))^2. \quad (3)$$

Here, $\{\mathbf{x}_n\}_{n=1}^{N_I}$ are collocation points sampled over the interior of the domain Ω , and $\{\mathbf{x}_i^{(l)}\}_{i=1}^{N_B^{(l)}}$ are collocation points sampled on the boundary subsets $\Gamma_l \subset \partial\Omega$, associated with boundary operators $\mathcal{B}_l$. The weights λ_l are hyperparameters used to balance the strength of each constraint in the optimization problem.

Thanks to the linearity of both the differential operator $\mathcal{N}$ and the boundary operators $\mathcal{B}_l$, the loss functional (3) is quadratic in the unknown coefficients $\mathbf{a}$. This allows us to reformulate the problem as a structured linear least-squares problem:

$$\min_{\mathbf{a} \in \mathbb{R}^{JK}} \|\mathbf{N}\mathbf{a} - \mathbf{f}\|^2 + \|\mathbf{B}\mathbf{a} - \mathbf{g}\|^2, \quad (4)$$

where $\mathbf{N}$ and $\mathbf{B}$ are the system matrices corresponding to the PDE and boundary residuals, and $\mathbf{f}$ and $\mathbf{g}$ are the vectors of evaluated right-hand side functions at the interior and boundary collocation points, respectively.

Equation (4) can be expressed as a single global least-squares problem given by

$$\min_{\mathbf{a} \in \mathbb{R}^{JK}} \|\mathbf{M}\mathbf{a} - \mathbf{h}\|^2, \quad (5)$$

where $\mathbf{M}$ and $\mathbf{h}$ are defined as

$$\mathbf{M} = \begin{bmatrix} \mathbf{N} \\ \mathbf{B} \end{bmatrix}, \quad \mathbf{h} = \begin{bmatrix} \mathbf{f} \\ \mathbf{g} \end{bmatrix}.$$

This global problem combines the residual minimization over the PDE and the boundary conditions into a single linear system, where each row block of $\mathbf{M}$ and $\mathbf{h}$ corresponds to either interior or boundary collocation points.

Indexing conventions and notations. Before discussing the structure of the system matrices, we introduce some convenient indexing conventions:

- Let $\ell = (j-1)K + k$ map the pair (j, k) from Equation (1) to a single global neuron index.
- Let $p = \sum_{k=1}^{l-1} N_B^{(k)} + i$ map the pair (i, l) from Equation (3) to a global index over boundary collocation points.
- Let $N_B = \sum_{l=1}^L N_B^{(l)}$ denote the total number of boundary points.

We now define the vectors and matrices involved in the least-squares problem (5):

- The vector $\mathbf{f} = \{\frac{1}{\sqrt{N_I}} f(\mathbf{x}_n)\}_{n=1}^{N_I}$ contains evaluations of the PDE right-hand side at interior collocation points.
- The vector $\mathbf{g} = \{g_p\}_{p=1}^{N_B}$, where $g_p = \sqrt{\frac{\lambda_l}{N_B^{(l)}}} g_l(\mathbf{x}_i^{(l)})$, contains evaluations of the boundary data at boundary collocation points.
- The parameter vector is $\mathbf{a} = \{a_\ell\}_{\ell=1}^{JK}$, where $a_\ell = a_{j,k}$.

Matrices $\mathbf{N} \in \mathbb{R}^{N_I \times JK}$ and $\mathbf{B} \in \mathbb{R}^{N_B \times JK}$ are defined component-wise by

$$N_{n,\ell} = \frac{1}{\sqrt{N_I}} \mathcal{N}[(\omega_j \phi_{jk})(\mathbf{x}_n)], \quad B_{p,\ell} = \sqrt{\frac{\lambda_l}{N_B^{(l)}}} \mathcal{B}_l[(\omega_j \phi_{jk})(\mathbf{x}_i^{(l)})],$$

where $\ell = (j-1)K + k$, $n = 1, \dots, N_I$, and $p = 1, \dots, N_B$.

Domain decomposition structure. Since the window functions ω_j have local support, the matrix $\mathbf{M} \in \mathbb{R}^{(N_I+N_B) \times JK}$ is sparse: the entry $M_{r,\ell}$ is nonzero only when $\omega_j(\mathbf{x}_r) \neq 0$.

A common problem in RFM is heavy linear dependence between the columns of $\mathbf{M}$ and consequently a large condition number. We want to exploit the sparsity of $\mathbf{M}$ to improve on this in an efficient way. To this end, we define the index sets

$$I_j = \{r \mid \omega_j(\mathbf{x}_r) \neq 0\}, \quad K_j = \{(j-1)K + 1, \dots, jK\},$$

where I_j corresponds to interior and boundary points influenced by subdomain j , and K_j to global neuron indices associated with that subdomain.

Let $\mathbf{M}_j = \mathbf{M}|_{I_j \times K_j}$ denote the submatrix of $\mathbf{M}$ restricted to $I_j \times K_j$. Introducing restriction matrices $\mathbf{V}_j \in \mathbb{R}^{|I_j| \times (N_I+N_B)}$ and $\mathbf{W}_j \in \mathbb{R}^{K \times JK}$, we express $\mathbf{M}_j$ as

$$\mathbf{M}_j = \mathbf{V}_j \mathbf{M} \mathbf{W}_j^T.$$

This yields the decomposition

$$\mathbf{M} = \sum_{j=1}^J \mathbf{V}_j^T \mathbf{M}_j \mathbf{W}_j. \quad (6)$$

This decomposition (6) highlights the localized, block-sparse structure of the system matrix $\mathbf{M}$ induced by the use of compactly supported window functions. An example plot of this block-sparse structure is shown in Figure 3. While this sparsity is beneficial for computational efficiency, it does not fully mitigate the adverse effects of feature redundancy and near-linear dependence among columns, which can lead to ill-conditioning of the system. To address this, several preconditioning strategies have been proposed in the literature that aim to exploit the domain-decomposed structure of $\mathbf{M}$.

2.2. Preconditioning the RFM via domain decomposition

In this section, we briefly review the preconditioner introduced in [20], which is based on the additive Schwarz method applied to the normal equations. Starting from the decomposition of the system matrix into localized contributions,

$$\mathbf{M} = \sum_{j=1}^J \mathbf{V}_j^T \mathbf{M}_j \mathbf{W}_j,$$

where $\mathbf{M}_j$ is the submatrix associated with subdomain Ω_j and $\mathbf{V}_j, \mathbf{W}_j$ are restriction operators as introduced in (6), the method proceeds by forming the Gramian matrix $\mathbf{A} = \mathbf{M}^T \mathbf{M}$. From this, one defines local Gramian blocks

$$\mathbf{A}_j = \mathbf{W}_j \mathbf{A} \mathbf{W}_j^T = \mathbf{M}_j^T \mathbf{M}_j,$$

which correspond to diagonal sub-blocks of $\mathbf{M}^T \mathbf{M}$ associated with each subdomain. After the application of a PCA to eliminate small singular values and reducing degrees of freedom, leading to a first improvement system conditioning, these blocks are used and assembled into the additive Schwarz preconditioner

$$\mathbf{A}_{\text{AS}}^{-1} = \sum_{j=1}^J \mathbf{W}_j \mathbf{A}_j^{-1} \mathbf{W}_j^T.$$

Although effective in improving convergence in certain settings, a key drawback of this approach is that it operates on the normal equations rather than the original least-squares problem. Forming the Gramian matrix $\mathbf{M}^T \mathbf{M}$ explicitly can lead to numerical difficulties: its condition number is the square of that of $\mathbf{M}$, i.e., $\kappa(\mathbf{M}^T \mathbf{M}) = \kappa(\mathbf{M})^2$, potentially exacerbating round-off errors and amplifying the effect of poorly scaled features.

To overcome the limitations associated with preconditioning the normal equations, we seek alternative strategies that operate directly on the original least-squares system. Beyond numerical stability, this shift is also driven by computational considerations: storing the Gramian matrix $\mathbf{M}^T \mathbf{M}$ can be costly, both in time and memory, and it typically destroys the sparsity pattern of $\mathbf{M}$. In contrast, the matrix $\mathbf{M}$ itself, due to the local support of the window functions, is often highly sparse, and preserving this sparsity is crucial for scalability in high-dimensional settings.

2.3. Rank revealing QR factorization

To exploit this structure efficiently, we turn to the rank-revealing QR (RRQR) factorization as an algebraic tool for dimension reduction and feature selection within each subdomain. Compared to singular value decomposition (SVD) also used in [20], RRQR offers a favorable trade-off: while both methods have $O(mn^2)$ complexity for a $m \times n$ matrix with $m \geq n$, RRQR is typically more numerically stable, faster in practice, amenable to sparse input, and returns actual columns of the matrix rather than linear combinations, enhancing interpretability and enabling localized filtering. In the next section, we introduce the formal definition of RRQR and outline how it forms the basis of our preconditioning approach.

The RRQR-based strategy introduced here filters out redundant features within each subdomain, leading to a preconditioner that acts directly on the original least-squares system while preserving sparsity. In our setting, the RRQR is applied locally to each matrix $\mathbf{M}_j$ rather than to the global matrix $\mathbf{M}$.

Given a matrix $\mathbf{P} \in \mathbb{R}^{m \times n}$ with $m \geq n$, a σ -rank-revealing QR (RRQR) factorization permutes the columns of $\mathbf{P}$ so that

$$\mathbf{P} \Pi = \mathbf{Q} \begin{pmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ & \mathbf{R}_{22} \end{pmatrix},$$

where $\mathbf{Q}$ has orthonormal columns, $\mathbf{R}_{11}$ is square and well-conditioned, and the trailing block $\mathbf{R}_{22}$ has norm at most σ . Algorithms such as the strong RRQR of Gu and Eisenstat [23] produce such a factorization, effectively identifying a well-conditioned basis for the dominant column space of $\mathbf{P}$ while discarding directions whose contribution is uniformly small.

This makes RRQR a natural tool for our local filtering step: it retains only the most informative features in each subdomain, reducing redundancy without compromising stability. Interestingly, in certain randomized settings – such as when $\mathbf{P}$ arises from features of randomly weighted neural networks – these well-conditioned bases also tend to emerge with high probability. Such probabilistic refinements are presented in Appendix A.

3. QR-based filtered RFM preconditioner

This section presents our main theoretical and algorithmic contribution: a subdomain-wise filtering strategy based on σ -rank-revealing QR (RRQR) factorizations, designed to address the ill-conditioning that arises in least-squares problems from Random Feature Methods (RFM) with overlapping subdomains. The method has two goals:

- **Remove local redundancy:** Within each subdomain, excess random features can become nearly linearly dependent, inflating the condition number of the local system.
- **Control overlap interference:** Even well-conditioned subdomains can interact unfavorably through overlap, further degrading the conditioning of the global system.

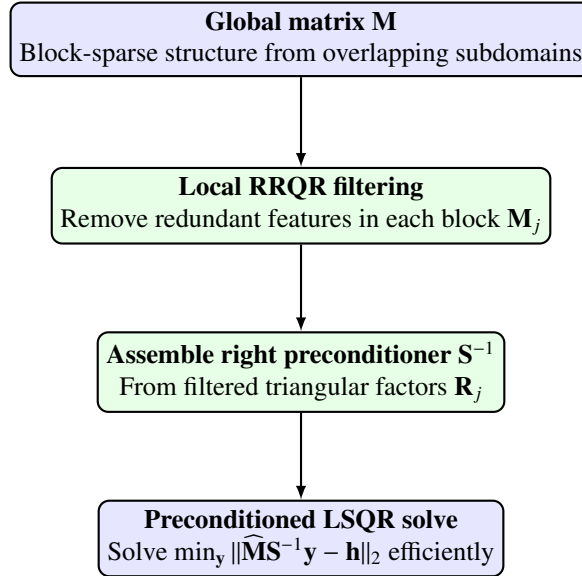


Figure 1: Workflow of the proposed RRQR filtering and preconditioning method.

Our approach applies RRQR locally to each subdomain matrix $\mathbf{M}_j$ to retain only the most informative columns—those corresponding to singular values above a threshold σ —thereby reducing redundancy while preserving sparsity. The triangular factors from these local decompositions are then assembled into a sparse right preconditioner $\mathbf{S}^{-1}$ for the global least-squares problem.

Figure 1 summarizes the overall process: starting from the block-sparse global matrix $\mathbf{M}$, we perform local RRQR filtering, assemble the preconditioner, and solve the preconditioned system efficiently with an iterative least-squares solver. A deterministic bound on the condition number of the preconditioned system, which captures the effect of overlap size on subdomain coupling, is given later in this section. Probabilistic refinements of this bound, using a random matrix model for the local QR factors, are presented in Appendix A.

We now describe the local filtering, assembly of the global preconditioner, least squares solver, and condition number estimates in detail.

3.1. Local filtering

The RRQR decomposition permutes the columns of a matrix to expose a well-conditioned leading block while ensuring that the trailing block is small in spectral norm. Applying this locally to each $\mathbf{M}_j$ allows us to identify a numerically full-rank basis for its column space, discard directions below the σ threshold, and keep the local sparsity pattern intact. The process is summarized in Algorithm 1.

Algorithm 1 σ -RRQR filtering method

Inputs: block-sparse matrix $\mathbf{M}$, subdomain count J , threshold $\sigma > 0$

for $j = 1, \dots, J$ **do**

 Compute σ -RRQR factorization of $\mathbf{M}_j$:

$$\mathbf{M}_j \Pi_j = \mathbf{Q}_j \begin{pmatrix} \mathbf{R}_j & * \\ & \mathbf{T}_j \end{pmatrix}$$

 Retain columns associated with $\mathbf{R}_j$ to form $\widehat{\mathbf{M}}_j$

end for

Assemble $\widehat{\mathbf{M}}$ from all $\widehat{\mathbf{M}}_j$

Output: global filtered matrix $\widehat{\mathbf{M}}$

Note in our algorithm, the threshold σ for each $\mathbf{M}_j$ is defined relative to the leading diagonal value of $\mathbf{R}_j$.

3.2. Assembly of the preconditioner

Recall from the domain decomposition identity (6) that the global matrix $\mathbf{M}$ can be written as

$$\mathbf{M} = \sum_{j=1}^J \mathbf{v}_j^T \mathbf{M}_j \mathbf{w}_j.$$

After applying local QR factorization and filtering to each subdomain, i.e., $\widehat{\mathbf{M}}_j = \widehat{\mathbf{Q}}_j \widehat{\mathbf{R}}_j$, we obtain the global filtered decomposition

$$\widehat{\mathbf{M}} = \sum_{j=1}^J \mathbf{v}_j^T \widehat{\mathbf{Q}}_j \widehat{\mathbf{R}}_j \widehat{\mathbf{W}}_j.$$

This suggests defining a preconditioner that operates directly on the original system and not on the normal equation without requiring full matrix assembly, hence being suitable for parallel computations. Specifically, we define the (right) preconditioner

$$\mathbf{S}^{-1} = \sum_{j=1}^J \widehat{\mathbf{W}}_j^T \widehat{\mathbf{R}}_j^{-1} \widehat{\mathbf{W}}_j.$$

Because each matrix $\widehat{\mathbf{R}}_j$ is upper triangular, applying $\mathbf{S}^{-1}$ involves only inexpensive back-substitution. Moreover, the preconditioner never needs to be explicitly formed, only local factors $\widehat{\mathbf{R}}_j$ and selection matrices $\widehat{\mathbf{W}}_j$ are needed.

Proposition 3.1. *The right preconditioned matrix simplifies to*

$$\mathbf{Q} := \widehat{\mathbf{M}} \mathbf{S}^{-1} = \sum_{j=1}^J \mathbf{v}_j^T \widehat{\mathbf{Q}}_j \widehat{\mathbf{W}}_j. \quad (7)$$

Proof. Substituting the factorizations $\widehat{\mathbf{M}}_j = \widehat{\mathbf{Q}}_j \widehat{\mathbf{R}}_j$ into the definition of $\widehat{\mathbf{M}}$ gives

$$\widehat{\mathbf{M}} \mathbf{S}^{-1} = \sum_{j=1}^J \mathbf{v}_j^T \widehat{\mathbf{Q}}_j \widehat{\mathbf{R}}_j \widehat{\mathbf{W}}_j \widehat{\mathbf{W}}_j^T \widehat{\mathbf{R}}_j^{-1} \widehat{\mathbf{W}}_j = \sum_{j=1}^J \mathbf{v}_j^T \widehat{\mathbf{Q}}_j \widehat{\mathbf{W}}_j,$$

using the identity $\widehat{\mathbf{R}}_j \widehat{\mathbf{R}}_j^{-1} = \mathbf{I}$ on the range of $\widehat{\mathbf{W}}_j$. □

This formulation isolates the orthogonal components $\widehat{\mathbf{Q}}_j$, which are numerically stable, and removes the influence of the potentially small singular values in $\widehat{\mathbf{R}}_j$, which would otherwise degrade the conditioning of the system. As a result, the preconditioned system is better suited for iterative solvers. Furthermore, the preconditioned system $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ has the same block structure and sparsity pattern as $\mathbf{M}$ filtered on its dominant columns (as shown in Figure 3). Given this block structure is sparse, this allows us to use efficient sparse linear solvers to solve the preconditioned system.

3.3. Least-squares solver

Having explained our filtering strategy and global preconditioner, our full workflow (as summarized by Figure 1) for solving the original least-squares system is shown in Algorithm 2.

Algorithm 2 ELM-FBPINN least-squares solver with RRQR filtering and preconditioning

Inputs: sparse matrix $\mathbf{M}$ and vector $\mathbf{h}$ in global least-squares problem $\|\mathbf{M}\mathbf{a} - \mathbf{h}\|^2$ (Equation (5)), number of subdomains J , relative threshold $\sigma > 0$

Compute σ -RRQR factorization, $\widehat{\mathbf{M}}$, of $\mathbf{M}$, filtering dominant columns, using Algorithm 1

Compute right-side preconditioned system $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ using Equation (7)

Solve preconditioned least-squares problem $\min_{\mathbf{y}} \|\widehat{\mathbf{M}}\mathbf{S}^{-1}\mathbf{y} - \mathbf{h}\|^2$ using a sparse linear solver (e.g. LSQR [24])

Compute least-squares solution for dominant columns $\hat{\mathbf{a}} = \mathbf{S}^{-1}\mathbf{y}$, using sparse back-substitution

Output: global least-squares solution, $\mathbf{a}$, with values

$$\mathbf{a} = \begin{cases} \hat{\mathbf{a}} & \text{at dominant columns} \\ 0 & \text{otherwise.} \end{cases}$$

3.4. Condition number estimate for overlapping subdomains

In this section we will analyze the spectral properties of the preconditioned matrix from Theorem 3.1 in the simplified case of J overlapping subdomains seen as consecutive blocks in one dimension or that may arise from a strip-wise decomposition in higher dimensions.

Let J be the number of overlapping subdomains. Each subdomain contributes a block of K features. The matrix $\mathbf{Q} \in \mathbb{R}^{N \times JK}$ is composed of column blocks $\widehat{\mathbf{Q}}_j$ corresponding to subdomains which are in turn subdivided into subblocks as follows:

- Each subdomain j contributes a local feature matrix $\mathbf{Q}_{jj} \in \mathbb{R}^{k_j \times K}$ for its interior,
- Overlapping regions between adjacent domains j and $j+1$ are represented by matrices $\mathbf{Q}_{j,j+1}, \mathbf{Q}_{j+1,j} \in \mathbb{R}^{\ell_j \times K}$,
- The columns of each $\widehat{\mathbf{Q}}_j$ are orthonormal.

Then $\mathbf{Q}$ has the block structure:

$$\mathbf{Q} = \begin{bmatrix} \mathbf{Q}_{11} & 0 & 0 & \cdots & 0 \\ \mathbf{Q}_{21} & \mathbf{Q}_{22} & 0 & \cdots & 0 \\ 0 & \mathbf{Q}_{32} & \mathbf{Q}_{33} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \mathbf{Q}_{JJ} \end{bmatrix} \in \mathbb{R}^{N \times JK} \quad (8)$$

Given that the columns of each $\widehat{\mathbf{Q}}_j$ are orthonormal and that columns of different subdomains are disjoint, we compute:

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_{JK} + \mathbf{E}$$

where $\mathbf{E}$ contains off-diagonal interactions due to overlaps. Specifically, $\mathbf{E}$ is a block tridiagonal matrix with:

$$\mathbf{E}_{j,j+1} = \mathbf{Q}_{j+1,j}^\top \mathbf{Q}_{j,j}, \quad \mathbf{E}_{j+1,j} = \mathbf{E}_{j,j+1}^\top$$

and all diagonal blocks are the identity:

$$(\mathbf{Q}^\top \mathbf{Q})_{jj} = \mathbf{I}_K \quad \text{for all } j.$$

So the matrix $\mathbf{Q}^\top \mathbf{Q}$ has the form:

$$\mathbf{Q}^\top \mathbf{Q} = \begin{bmatrix} \mathbf{I} & \mathbf{A}_1 & 0 & \cdots & 0 \\ \mathbf{A}_1^\top & \mathbf{I} & \mathbf{A}_2 & \ddots & \vdots \\ 0 & \mathbf{A}_2^\top & \mathbf{I} & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \mathbf{A}_{J-1} \\ 0 & \cdots & 0 & \mathbf{A}_{J-1}^\top & \mathbf{I} \end{bmatrix}$$

with $\mathbf{A}_j := \mathbf{Q}_{j+1,j}^\top \mathbf{Q}_{j,j} \in \mathbb{R}^{K \times K}$.

Theorem 3.2 (Condition Number Bound for the Overlapping Block Matrix). *Let $\mathbf{Q} \in \mathbb{R}^{N \times JK}$ be a block-structured matrix constructed from J overlapping subdomains with orthonormal columns defined as in Equation (8). Assume further that:*

$$\|\mathbf{A}_j\|_2 \leq \alpha \quad \text{for all } j.$$

Then the condition number of $\mathbf{Q}$ satisfies:

$$\kappa(\mathbf{Q}) \leq \left(\frac{1+2\alpha}{1-2\alpha} \right)^{1/2}, \quad \text{provided } 2\alpha < 1.$$

Proof. By the symmetry of $\mathbf{E}$, its eigenvalues are real. Using the definition, the submultiplicativity of the spectral norm and Cauchy-Schwarz inequality, we can easily estimate norm of $\mathbf{E}$:

$$\|\mathbf{E}\|_2 \leq 2\alpha.$$

Thus, the eigenvalues λ of $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I} + \mathbf{E}$ satisfy

$$1 - 2\alpha \leq \lambda \leq 1 + 2\alpha.$$

Therefore, the condition number of $\mathbf{Q}$ in the spectral norm is

$$\kappa(\mathbf{Q}) = \sqrt{\kappa(\mathbf{Q}^\top \mathbf{Q})} \leq \left(\frac{1+2\alpha}{1-2\alpha} \right)^{1/2},$$

as long as $1 - 2\alpha > 0$, i.e., $2\alpha < 1$. □

Remark 3.3. *The matrix $\mathbf{Q}^\top \mathbf{Q}$ is always symmetric and positive semidefinite, since it is a Gram matrix. All its eigenvalues are non-negative.*

However, the bound above is only meaningful when $\mathbf{Q}^\top \mathbf{Q}$ is strictly positive definite and a stricter inequality holds. Since we estimate $\lambda_{\max}(\mathbf{E}) \leq 2\alpha$, the condition $2\alpha < 1$ ensures invertibility of $\mathbf{Q}^\top \mathbf{Q}$ and validity of the condition number estimate.

The value $\alpha = \max_j \|\mathbf{Q}_{j+1,j}^\top \mathbf{Q}_{j,j}\|_2$ measures the strength of coupling between adjacent subdomains. The smaller the overlap-induced interactions, the better conditioned the global system is — which is somewhat counterintuitive from the domain decomposition perspective.

Remark 3.4. *In the special case of only two overlapping blocks ($J = 2$), the matrix $\mathbf{Q}^\top \mathbf{Q}$ has the form:*

$$\mathbf{Q}^\top \mathbf{Q} = \begin{bmatrix} \mathbf{I} & \mathbf{A}_1 \\ \mathbf{A}_1^\top & \mathbf{I} \end{bmatrix},$$

The eigenvalues of $\mathbf{Q}^\top \mathbf{Q}$ are then $1 \pm \sigma_i(\mathbf{A}_1)$, and the condition number becomes:

$$\kappa(\mathbf{Q}) = \left(\frac{1 + \|\mathbf{A}_1\|_2}{1 - \|\mathbf{A}_1\|_2} \right)^{1/2}, \quad \text{valid whenever } \|\mathbf{A}_1\|_2 < 1.$$

Hence, the condition for positive definiteness is simply $\|\mathbf{A}_1\|_2 < 1$, which is strictly weaker than $2\alpha < 1$. This makes the two-block case less restrictive and ensures well-conditioning as long as we have positive definiteness $\mathbf{Q}^\top \mathbf{Q} > 0$.

Theorem 3.2 shows that, in the worst case, the conditioning of the preconditioned system depends only on the strength of nearest-neighbour interactions between subdomains, measured by α . If these interactions are weak (small α), the global system remains well conditioned regardless of the number of subdomains or total features. Conversely, large overlap couplings drive α towards the $2\alpha < 1$ threshold, causing rapid growth in $\kappa(\mathbf{Q})$.

The RRQR filtering directly influences α . Recall that $\mathbf{A}_j = \mathbf{Q}_{j,j+1}^\top \mathbf{Q}_{j,j}$ involves basis vectors from two adjacent subdomains, restricted to the overlap region. When a σ -RRQR is applied to each local matrix $\mathbf{M}_j$, all columns associated with singular values smaller than σ are discarded. This eliminates directions that are both weakly represented locally and highly correlated with features from the neighbouring subdomain. Consequently, the remaining columns $\widehat{\mathbf{M}}_j$ — and their orthonormal factors $\widehat{\mathbf{Q}}_j$ — exhibit reduced cross-interface correlation, leading to smaller $\|\mathbf{A}_j\|_2$. In qualitative terms, a larger σ produces a more aggressively filtered basis, which reduces α and thus tightens the bound in Theorem 3.2, at the cost of discarding more features. Quantitative estimates of $\|\mathbf{A}_j\|_2$ are obtained in Appendix A using probabilistic models; a deeper theoretical analysis of these couplings lies beyond the scope of the present paper.

4. Numerical experiments

In this section we assess the numerical performance of our proposed subdomain RRQR filtering and preconditioning strategy. We study how it affects the convergence, efficiency, and accuracy of ELM-FBPINNs when solving boundary and initial value problems (BVPs and IVPs) in one, two and three dimensions. We are particularly interested in how it improves the conditioning of the least-squares problem given by Equation (5) and its subsequent solution accuracy.

This section is structured as follows. First, in Section 4.1, we describe the problems studied in detail. Next, in Section 4.2, we explain the common implementation details across all experiments. Then, in Section 4.3 we describe the baseline models we compare to. Finally, in Section 4.4 we describe our numerical results in detail.

4.1. Problems studied

This section describes the BVPs and IVPs studied. For each problem, we carry out ablation studies of the ELM-FBPINN with RRQR filtering. We assess how its accuracy and convergence changes with different σ values in the RRQR filtering, depths of subdomain networks, and subdomain network activation functions. We also carry out strong and weak scaling studies, where we assess how the total number of subdomains, J in Equation (1), the degree of overlap between subdomains, and number of basis functions, K in Equation (1), affects performance, and how our method performs when the solution complexity increases.

4.1.1. Harmonic oscillator in 1D

The first problem we study is the damped harmonic oscillator in one dimension. This models the displacement, $u(t)$, of a mass on a spring through time, and is given by

$$\begin{aligned} m \frac{d^2 u}{dt^2} + \mu \frac{du}{dt} + ku &= 0 \quad \text{in } \Omega = [0, 1], \\ u(0) &= 1, \\ \frac{du}{dt}(0) &= 0, \end{aligned} \tag{9}$$

where m is the mass of the oscillator, μ is the coefficient of friction, and k is the spring constant. We focus on solving the problem in the under-damped state, which occurs when

$$\delta < \omega_0, \quad \text{where } \delta = \frac{\mu}{2m}, \quad \omega_0 = \sqrt{\frac{k}{m}}. \tag{10}$$

In this case, the exact solution is known and is given by

$$u(t) = e^{-\delta t} (2A \cos(\phi + \omega t)), \quad \text{with } \omega = \sqrt{\omega_0^2 - \delta^2}, \tag{11}$$

where A and ϕ can be derived from the initial conditions. For all cases studied, we fix $\mu = 4$, and $m = 1$, and when performing weak scaling studies we increase the solution complexity by increasing ω_0 .

4.1.2. Multi-scale Laplacian in 2D

The second problem we study is a multi-scale Laplacian problem in two dimensions, as defined in [15], given by

$$\begin{aligned} -\Delta u &= f \quad \text{in } \Omega = [0, 1]^2, \\ u &= 0 \quad \text{on } \partial\Omega, \end{aligned} \quad (12)$$

with a source term given by

$$f(x_1, x_2) = \frac{2}{n} \sum_{i=1}^n (\omega_i \pi)^2 \sin(\omega_i \pi x_1) \sin(\omega_i \pi x_2). \quad (13)$$

Then, the exact solution is given by

$$u(x_1, x_2) = \frac{1}{n} \sum_{i=1}^n \sin(\omega_i \pi x_1) \sin(\omega_i \pi x_2). \quad (14)$$

In this study we fix $\omega_i = 2^i$. The solution to this problem is highly multi-scale, with the number of multi-scale components controlled by the parameter n . When performing weak scaling studies we increase the solution complexity by increasing n .

Hard boundary constraints. For this problem, we use a hard-constraining approach, as originally described by [3], to assert the boundary conditions in the ELM-FBPINN. Specifically, we assume that the solution has the following ansatz

$$\tilde{u}(x_1, x_2, \mathbf{a}) = \tanh(\omega_n x_1) \tanh(\omega_n (1 - x_1)) \tanh(\omega_n x_2) \tanh(\omega_n (1 - x_2)) \hat{u}(x_1, x_2, \mathbf{a}), \quad (15)$$

with $\hat{u}$ as defined in Equation (1). It is simple to show that this ansatz automatically obeys the boundary condition in Equation (12). This has the advantage that when training the ELM-FBPINN the boundary losses in Equation (3) can be removed. It is important to note that when using this ansatz Equation (3) remains quadratic in $\mathbf{a}$, which still ensures we can form the least-squares system defined in Equation (5).

4.1.3. Time-varying wave equation in (2+1)D

The final problem studied is the time-varying two dimensional acoustic wave equation with Dirichlet boundary conditions, given by

$$\begin{aligned} \nabla^2 u - \frac{1}{c^2} \frac{\partial^2 u}{\partial t^2} &= 0 \quad \text{in } \Omega = [-1, 1]^2 \times [0, 1], \\ u &= 0 \quad \text{on } \partial\Omega_{\mathbf{x}} \times [0, 1], \\ u(\mathbf{x}, t=0) &= e^{-\frac{1}{2}(\|\mathbf{x}-\boldsymbol{\mu}\|/\lambda)^2}, \\ \frac{\partial u}{\partial t}(\mathbf{x}, t=0) &= 0, \end{aligned} \quad (16)$$

where $c \in \mathbb{R}^1$ is a constant wave speed, and $\boldsymbol{\mu} \in \mathbb{R}^2$ and $\lambda \in \mathbb{R}^1$ control the starting location and central frequency of a stationary Gaussian point source. For all problems studied we set $c = 1$ and $\boldsymbol{\mu} = (0, 0)$.

With this setup the solution is a circular wavefront expanding from the center of the domain and reflecting from the boundaries. We are not aware of an exact solution, and therefore compare our results to a high-fidelity finite difference acoustic wave equation solver, described in detail in Appendix B. When performing weak scaling studies we increase the solution complexity by decreasing λ which increases the solution frequency.

Hard boundary constraints. Similar to the multi-scale Laplacian problem in Section 4.1.2, we use a hard boundary constraint when solving this problem, assuming the solution has the following ansatz

$$\tilde{u}(x_1, x_2, t, \mathbf{a}) = \tanh(x_1/d) \tanh((1 - x_1)/d) \tanh(x_2/d) \tanh((1 - x_2)/d) \quad (17)$$

$$\times \left(e^{-\frac{1}{2}((0.6t/\tau)^2 + \|\mathbf{x}-\boldsymbol{\mu}\|/\lambda)^2} + \tanh^2(t/\tau) \hat{u}(x_1, x_2, t, \mathbf{a}) \right), \quad (18)$$

where $d = \lambda/2.5$ and $\tau = \lambda/(2.5c)$. This ansatz is designed so that all boundary conditions in Equation (16) are automatically obeyed. It is also designed such that at times $t \gg \tau$, the initial Gaussian point source term is negligibly zero. This is done so that the ELM-FBPINN does not need to correct for this part of the ansatz once the point source has expanded away from its starting location.

4.2. Common implementation details

The ELM-FBPINNs with RRQR filtering tested share many common implementation details across the problems studied which are described below.

Domain decomposition. All problems are defined on rectangular domains such that $\Omega = \prod_{i=1}^d [a_i, b_i]$, where a_i and b_i are the edges of the domain along each input dimension. For simplicity, all models tested use a regular overlapping rectangular domain decomposition, defined as $D = \{\Omega_{\mathbf{j}}\}$ with $\mathbf{j} \in \{1, \dots, S\}^d$, where d is the number of input dimensions, S is the number of subdomains along each input dimension, $J = S^d$ is the total number of subdomains, and

$$\Omega_{\mathbf{j}} = \prod_{i=1}^d \Omega_{j_i}, \quad \text{where} \quad \Omega_{j_i} = \left[a_i + (b_i - a_i) \frac{(j_i - 1) - \delta/2}{S - 1}, a_i + (b_i - a_i) \frac{(j_i - 1) + \delta/2}{S - 1} \right], \quad (19)$$

where δ is defined as the *overlap ratio*. Note that an overlap ratio of less than 1 means that the subdomains are no longer overlapping. Example domain decompositions used in 1D and 2D are shown in Figure 2 and Figure 9.

As discussed in Section 2.1, window functions with local support are used to confine each ELM-FBPINN subdomain network to its subdomain. All models tested use the following window function for each subdomain

$$\omega_{\mathbf{j}} = \frac{\hat{\omega}_{\mathbf{j}}}{\sum_{\mathbf{j}} \hat{\omega}_{\mathbf{j}}}, \quad \text{where} \quad \hat{\omega}_{\mathbf{j}}(\mathbf{x}) = \prod_{i=1}^d [1 + \cos(\pi(x_i - \mu_{j_i}/\sigma_{j_i}))]^2, \quad (20)$$

such that the window functions form a partition of unity, where μ_{j_i} and σ_{j_i} are the center and half-width of each subdomain along each input dimension.

Network architecture. All models tested use a fully-connected neural network for each subdomain network, with the same size network in each subdomain. All hidden subdomain network parameters are randomly initialized using uniform LeCun initialization. Furthermore, the $\mathbf{x}$ inputs to each subdomain network are normalized to the range $[-1, 1]$ along each dimension over their individual subdomains.

Interior collocation points. For all models tested, we choose the number of interior collocation points for the PDE residual term in Equation (3) such that the global matrix $\mathbf{M}$ is always well-determined (square or otherwise tall). Specifically, we sample interior collocation points on a regular grid over the entire domain Ω with $S \cdot \text{ceil}(K^{\frac{1}{d}})$ collocation points along each input dimension. The total number of interior collocation points (which each contribute a row in $\mathbf{M}$) is therefore $[S \cdot \text{ceil}(K^{\frac{1}{d}})]^d \geq S^d K = JK$, where we note that JK is the total number of columns in the global $\mathbf{M}$ matrix.

Least-squares solver. All models tested use Algorithm 2 with the iterative LSQR solver [24] to solve the global least-squares problem (Equation (5)). This solver operates directly on the least-squares problem and avoids potential pitfalls associated with normal-equation-based solution approaches. We purposefully run the solver over 10,000 iterations without using a stopping condition, so that the full convergence behavior of each model can be displayed. In practice, the majority of models tested converge with much less iterations and a stopping condition can be used to significantly accelerate training times. When reporting the final error of a solve, we report the minimum error observed over all iteration steps.

Performance evaluation. All models are evaluated using the normalized L1 test accuracy, given by $e_{L_1}(\mathbf{a}) = \frac{1}{M} \sum_i^M \|\hat{u}(\mathbf{x}_i, \mathbf{a}) - u(\mathbf{x}_i)\|/\sigma$, where M is the number of test points and σ is the standard deviation of the set of true solutions $\{u(\mathbf{x}_i)\}_i^M$. Test points are sampled on a regular grid over the entire domain Ω . For robustness, all models are trained five times using different random seeds. Test accuracies are reported as the median value $\pm$ the range (max – min), while training times are reported as the mean $\pm$ one standard deviation.

Software and hardware implementation. We implement ELM-FBPINNs with RRQR filtering by building upon the highly optimized FBPINN JAX library available here: github.com/benmoseley/FBPINNs, described in more detail in [7, 15, 25]. We define sparse linear solvers using the SciPy library [26], and all models are trained on a single AMD EPYC 7742 CPU core with 50 GB RAM.

4.3. Baselines

We compare our ELM-FBPINNs with RRQR filtering to the following baselines:

1. PINNs [3, 4], optimizing all parameters of a single global neural network to minimize Equation (3) with gradient descent using the Adam optimizer and a learning rate of 1×10^{-3} ,
2. FBPINNs [7], optimizing all subdomain network parameters in Equation (1) (rather than freezing hidden parameters) to minimize Equation (3) with gradient descent using the Adam optimizer and a learning rate of 1×10^{-3} ,
3. ELM-FBPINNs without preconditioning, solving the global least-squares problem (Equation (5)) directly with sparse LSQR,
4. ELM-FBPINNs without preconditioning, solving the global least-squares problem (Equation (5)) via its normal equations with the conjugate gradient (CG) method,
5. ELM-FBPINNs with additive Schwarz (AS) preconditioning proposed by [20], solving the global least-squares problem (Equation (5)) via its normal equations with AS preconditioning without PCA filtering and the CG method.

For fairness, when comparing these models to ELM-FBPINNs with RRQR filtering, all relevant hyperparameters are kept the same, including problem definition, domain decomposition, subdomain network architecture, parameter initialization strategy, random seed, collocation point definition, loss function, and testing point definition. For the PINN and FBPINN baselines we optimize all network parameters. This turns Equation (3) into a nonlinear optimization problem and we therefore use gradient descent with 10,000 steps to optimize them. Furthermore, given PINNs do not use a domain decomposition, a larger global neural network with more hidden layers and hidden units is used for these models, and their inputs are normalized to the range $[-1, 1]$ along each dimension over the global domain.

4.4. Results

In this section we discuss the performance of ELM-FBPINNs with RRQR filtering on the problems described in Section 4.1.

4.4.1. Harmonic oscillator in 1D

Baseline model. We first study the harmonic oscillator problem with $\omega_0 = 60$. We train a ELM-FBPINN with RRQR filtering with $J = S = 20$, $\delta = 2.9$, $K = 8$, a subdomain network depth of $h = 1$, and tanh activation function, using $\sigma = 1 \times 10^{-8}$ during σ -RRQR filtering, and test the model with $50 \times S = 1000$ collocation points. The true solution, domain decomposition, subdomain network basis functions and model solution are shown in Figure 2. We find that the model is able to accurately predict the exact solution, with a normalized L1 test accuracy of $1.0 \pm 1.2 \times 10^{-4}$. During σ -RRQR filtering, we find that 8% of columns in $\mathbf{M}$ are discarded, and the condition number of $\mathbf{M}$ decreases from 2.9×10^{16} to 3.0×10^{11} . The condition number of the preconditioned matrix $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ further drops to 5.2×10^5 . This shows that both our filtering and preconditioning steps significantly improve the conditioning of the least-squares problem. The global matrix $\mathbf{M}$ and preconditioned matrix $\widehat{\mathbf{M}}\mathbf{S}^{-1}$, and their SVD spectra, are plotted in Figure 3, which shows the expected block-sparsity pattern of these matrices, and a reduction in dynamic range of their SVD spectra after preconditioning. The convergence of the LSQR solver is shown in Figure 4. We find that the model converges within 1000 iterations, whilst the total time taken to carry out the full 10,000 iterations is 4.0 ± 0.1 s.

Baseline comparison. Next, we compare our baseline ELM-FBPINN with RRQR filtering above to our baseline models. A comparison of the relevant condition numbers, test accuracy and total training time is shown in Table 1. Our model has the highest test accuracy, with the ELM-FBPINN with AS preconditioning and ELM-FBPINNs without preconditioning achieving a slightly lower test accuracy. The FBPINN and PINN baselines perform significantly worse than the models with linear solvers, being one and three orders of magnitude less accurate respectively, possibly because of the non-convexity of their loss function. All models with linear solvers

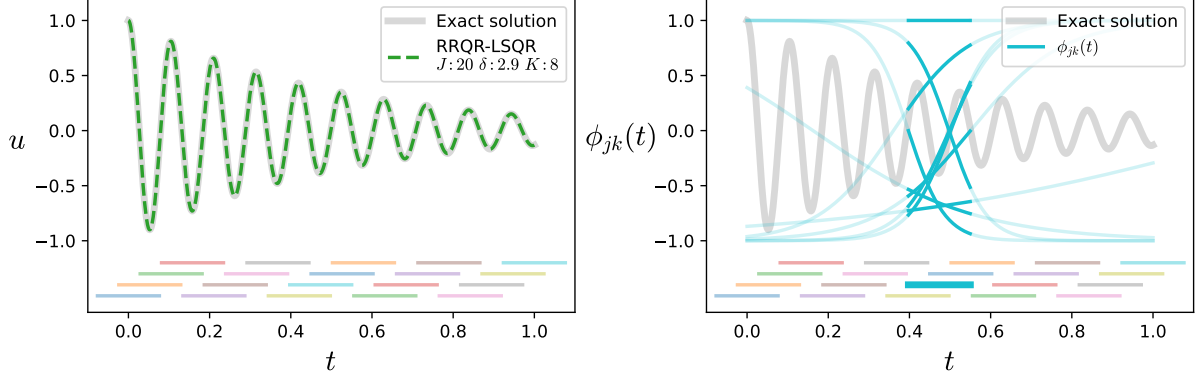


Figure 2: Exact solution (left), domain decomposition (both), model basis functions for the 10th subdomain (right), and model solution (left) when using the ELM-FBPINN with RRQR filtering to solve the harmonic oscillator in 1D.

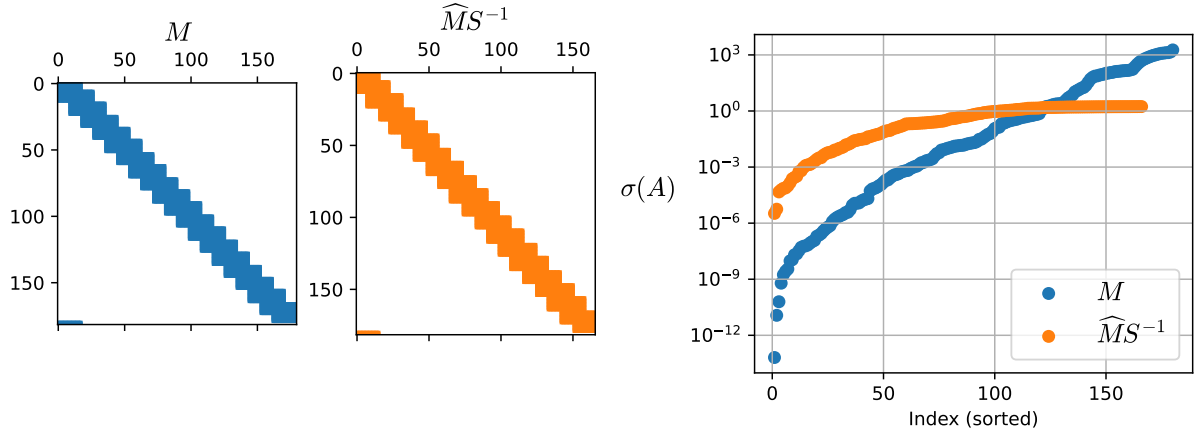


Figure 3: Sparsity plot of $\mathbf{M}$ and $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ (left) and their SVD spectra (right), when using the ELM-FBPINN with RRQR filtering to solve the harmonic oscillator in 1D.

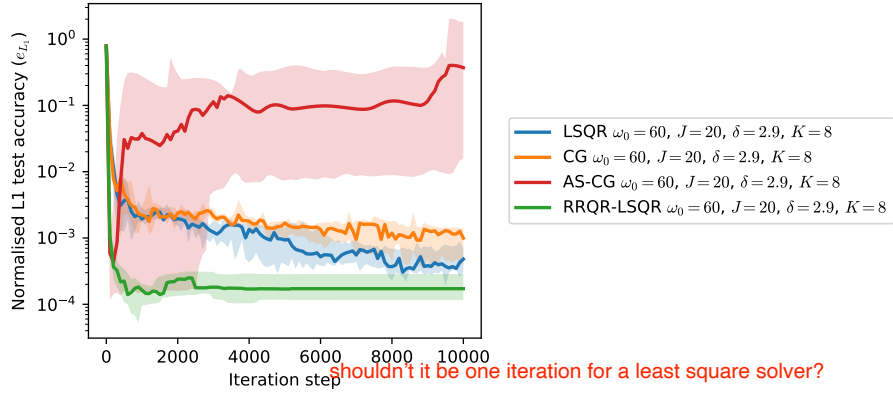


Figure 4: Convergence curves of the ELM-FBPINN with RRQR filtering and our baseline models when solving the harmonic oscillator in 1D.

have similar training times, whilst the FBPINN and PINN baselines are and nearly an order of magnitude slower to train due to the higher computational cost of gradient descent. The convergence curves of all linear solver models are compared in Figure 4. We find that the ELM-FBPINNs without preconditioning take the full 10,000 iteration steps to converge, whilst both our model and the ELM-FBPINN with AS preconditioning converge within 1000 steps. This shows that preconditioning is essential for enabling the solver to converge quickly. We notice that whilst the ELM-FBPINN with AS preconditioning converges quickly, it quickly becomes unstable at later iteration steps. Thus, it appears in practice a good stopping condition is required to use this model.

Network	(h, K)	$\kappa(\mathbf{M})$	$\kappa(\widehat{\mathbf{M}})$	$\kappa(\widehat{\mathbf{M}}\mathbf{S}^{-1})$	$\kappa(\mathbf{M}^T\mathbf{M})$	$\kappa(\mathbf{A}_{AS}^{-1}\mathbf{A})$	Optimiser	σ (ϕ_{jk} drop %)	e_{L_1}	Time (s)
FCN	(2, 64)	N/A	N/A	N/A	N/A	N/A	PINN-Adam	N/A (N/A)	$4.8 \pm 0.6 \times 10^{-1}$	$1.4 \pm 0.0 \times 10^1$
FCN	(1, 8)	N/A	N/A	N/A	N/A	N/A	FBPINN-Adam	N/A (N/A)	$3.5 \pm 1.5 \times 10^{-3}$	8.9 ± 0.1
ELM	(1, 8)	2.9×10^{16}	N/A	N/A	N/A	N/A	LSQR	N/A (N/A)	$2.9 \pm 1.8 \times 10^{-4}$	3.4 ± 0.0
ELM	(1, 8)	2.9×10^{16}	N/A	N/A	9.9×10^{18}	N/A	CG	N/A (N/A)	$6.0 \pm 9.2 \times 10^{-4}$	2.9 ± 0.0
ELM	(1, 8)	2.9×10^{16}	N/A	N/A	9.9×10^{18}	3.2×10^{12}	AS-CG	N/A (N/A)	$2.7 \pm 7.5 \times 10^{-4}$	3.6 ± 0.0
ELM	(1, 8)	2.9×10^{16}	3.0×10^{11}	5.2×10^5	N/A	N/A	RRQR-LSQR	1e-08 (8%)	$1.0 \pm 1.2 \times 10^{-4}$	4.0 ± 0.1
ELM	(1, 8)	2.9×10^{16}	2.8×10^{12}	3.7×10^6	N/A	N/A	RRQR-LSQR	1e-10 (2%)	$5.9 \pm 9.0 \times 10^{-5}$	4.1 ± 0.1
ELM	(1, 8)	2.9×10^{16}	1.6×10^8	1.4×10^4	N/A	N/A	RRQR-LSQR	1e-06 (21%)	$1.1 \pm 3.1 \times 10^{-4}$	4.1 ± 0.1
ELM	(1, 8)	2.9×10^{16}	2.7×10^5	2.4×10^2	N/A	N/A	RRQR-LSQR	1e-04 (43%)	$1.0 \pm 0.7 \times 10^{-3}$	4.0 ± 0.1
ELM	(1, 8)	2.9×10^{16}	2.6×10^3	5.9×10^1	N/A	N/A	RRQR-LSQR	1e-02 (67%)	$1.3 \pm 1.1 \times 10^{-2}$	4.2 ± 0.1
ELM- σ	(1, 8)	4.9×10^{17}	1.8×10^4	5.0×10^3	N/A	N/A	RRQR-LSQR	1e-08 (22%)	$2.0 \pm 2.3 \times 10^{-4}$	4.2 ± 0.3
ELM-sin	(1, 8)	6.4×10^{16}	1.1×10^{10}	3.2×10^3	N/A	N/A	RRQR-LSQR	1e-08 (23%)	$2.0 \pm 2.1 \times 10^{-4}$	4.1 ± 0.1
ELM	(3, 8)	2.4×10^{15}	4.5×10^{11}	6.5×10^5	N/A	N/A	RRQR-LSQR	1e-08 (3%)	$4.9 \pm 9.4 \times 10^{-5}$	4.7 ± 0.1
ELM	(5, 8)	7.6×10^{15}	2.6×10^{11}	1.7×10^5	N/A	N/A	RRQR-LSQR	1e-08 (12%)	$5.7 \pm 7.9 \times 10^{-5}$	5.3 ± 0.1

Table 1: Comparison of test error, total training time, and conditioning numbers for different models solving the harmonic oscillator in 1D.

Ablation tests. Next, we perform ablation tests on our baseline ELM-FBPINN with RRQR filtering. Specifically, we test its performance whilst varying σ between 1×10^{-10} , 1×10^{-6} , 1×10^{-4} , and 1×10^{-2} , using sigmoid and sinusoidal activation functions, and with a subdomain network depth of $h = 3$ and 5. The test accuracy and total training time of these models is reported in Table 1. As expected, the number of columns dropped by σ -RRQR filtering increases as σ increases, with 67% of columns dropped for $\sigma = 1 \times 10^{-2}$. Also as expected, the condition numbers of $\widehat{\mathbf{M}}$ and $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ decrease as σ increases, as more numerically dependent columns are dropped during filtering. However, the test accuracy decreases as σ increases, possibly because the model has less representational capacity as columns (basis functions) are dropped. Thus it appears important to balance good conditioning of the system against test time accuracy. We find that using sigmoid and sinusoidal activation functions does not improve test accuracy, whilst increasing h does improve test accuracy and reduces the condition number of $\mathbf{M}$, possibly because increasing the depth of the subdomain networks allows the basis functions to be more flexible and therefore less linearly dependent.

Strong scaling tests. Next we perform strong scaling tests, where the general principle is to test *fixing the problem complexity and increasing the model capacity*. In the ideal case, the model will improve its test accuracy in proportion with model capacity. For this problem, we fix the problem complexity by keeping $\omega_0 = 60$, as tested above. We then increase the model capacity in two different ways; first, by varying K between 4, 8, 16, 32, and 64, and second by fixing $K = 8$, but simultaneously varying S between 10, 20, 40, 80, and 160 and δ between 1.45, 2.9, 5.8, 11.6, and 23.2. Scaling S and δ in this way keeps the subdomain width fixed at 0.153 for all tests, but adds more subdomains with higher overlap. All other hyperparameters are kept the same as our baseline model above. Plots of the convergence curves and test accuracy versus total training time of the ELM-FBPINN with RRQR filtering, ELM-FBPINN without preconditioning with LSQR, and ELM-FBPINN with AS preconditioning for each scaling test are shown in Figure 5 and Figure 6. For all solvers, we find that the test accuracy improves when increasing K , showing good scaling behavior. However, the ELM-FBPINN without preconditioning with LSQR and ELM-FBPINN with AS preconditioning have significantly worse test accuracy than our model for large values of K . This is likely because the condition number of $\mathbf{M}$ increases significantly with K , and our model is the only model which filters basis functions (i.e. lowers K) before solving the system. Furthermore, our model is an order of magnitude faster than the ELM-FBPINN without preconditioning with LSQR, and two orders of magnitude faster than the ELM-FBPINN with AS preconditioning. This is both because our model drops columns before solving the system, which reduces the size of $\mathbf{M}$, and because computation of the $\mathbf{A}_{AS}^{-1}$ preconditioner is significantly more expensive in memory and time than our $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ preconditioner. For all solvers, we find that the test accuracy improves when simultaneously increasing S and δ , where in this case the ELM-FBPINN with AS preconditioning achieves a slightly higher test accuracy than our model for the final $S = 160, \delta = 23.2$ case, but does take significantly longer to do so.

Weak scaling test. Related to strong scaling tests, we also perform weak scaling tests, where the general principle is to test *increasing the problem complexity whilst proportionally increasing the model capacity*. In the ideal case, test accuracy will stay constant as the problem complexity and model capacity is increased. For this problem, we increase the problem complexity by varying ω_0 between 30, 60, 120, 240, and 580, whilst simultaneously increasing S between 10, 20, 40, 80, and 160. All other hyperparameters are kept the same as our baseline model above.

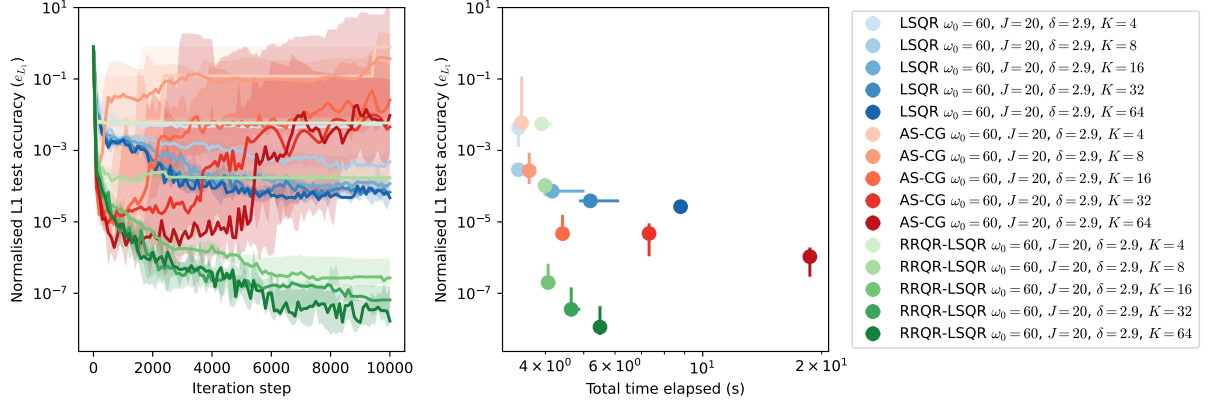


Figure 5: Strong scaling test for the harmonic oscillator in 1D, fixing the solution frequency, ω_0 , and varying the number of basis functions, K .

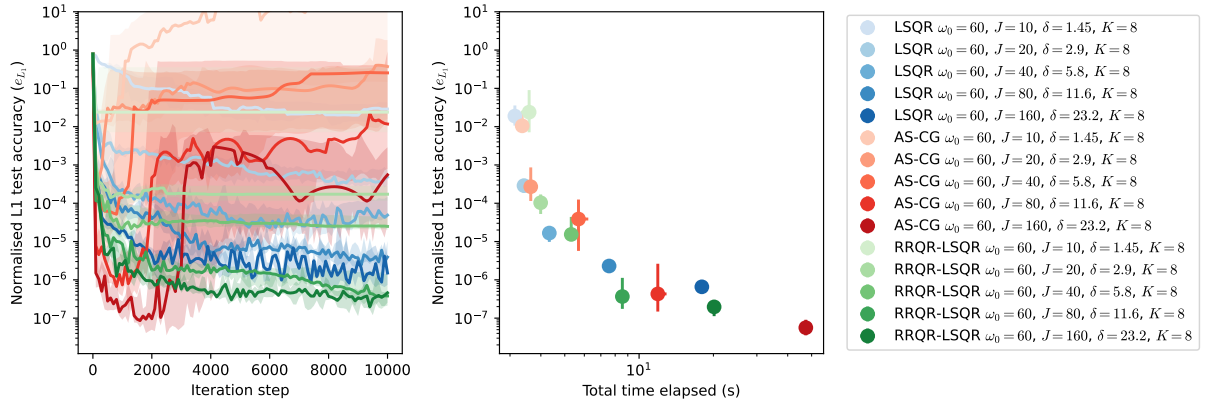


Figure 6: Strong scaling test for the harmonic oscillator in 1D, fixing the solution frequency, ω_0 , and simultaneously varying the number of subdomains, J , and the overlap between subdomains, δ .

Plots of the convergence curves and test accuracy versus total training time of the ELM-FBPINN with RRQR filtering, ELM-FBPINN without preconditioning with LSQR, and ELM-FBPINN with AS preconditioning for each case are shown in Figure 7, and the exact solution and ELM-FBPINN with RRQR filtering solution are plotted for different ω_0 values in Figure 8. We find that our model and the ELM-FBPINN with AS preconditioning have good scaling behavior, with test accuracy roughly staying constant as ω_0 increases, whilst the ELM-FBPINN without preconditioning with LSQR has poor scaling behavior, being unable to accurately model the $\omega_0 = 240$ and 580 solutions with the solver converging extremely slowly. This presents further evidence that preconditioning is essential especially for larger numbers of subdomains.

4.4.2. Multi-scale Laplacian in 2D

Baseline model. Next, we move on to studying the multi-scale Laplacian problem in 2D. As above, we first train a baseline ELM-FBPINN with RRQR filtering, studying the problem with $n = 3$, using $S = 16$, $\delta = 2.9$, $K = 16$, a subdomain network depth of $h = 1$, tanh activation function, $\sigma = 1 \times 10^{-8}$, and $6 \times S = 96$ collocation points along each input dimension for testing. The true solution, domain decomposition, and model solution are shown in Figure 9. Similar to the 1D harmonic oscillator problem, we find that the model is able to accurately predict the exact solution, with a normalized L1 test accuracy of $1.9 \pm 1.3 \times 10^{-3}$. Interesting, we find that no columns are dropped during σ -RRQR filtering, likely because the condition number of $\mathbf{M}$ is already relatively low at 1.4×10^9 . After preconditioning the condition number of $\mathbf{M}\mathbf{S}^{-1}$ further drops to 2.8×10^4 , and the SVD spectra of $\mathbf{M}$ and $\mathbf{M}\mathbf{S}^{-1}$ are plotted in Figure 9. The convergence of the LSQR solver is shown in Figure 10. We find that the model converges within a few hundred steps, whilst the total time taken to carry out the full 10,000 iterations is $3.1 \pm 0.1 \times 10^1$ s.

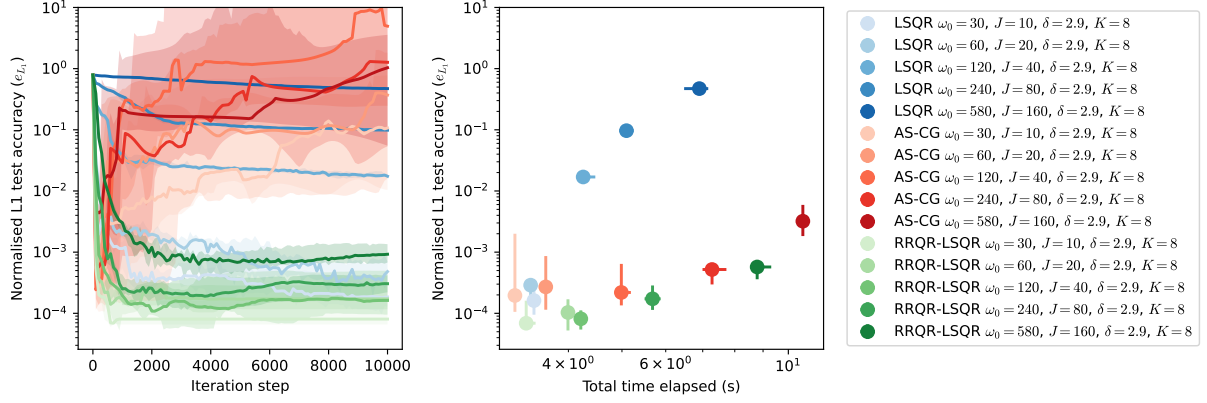


Figure 7: Weak scaling test for the harmonic oscillator in 1D, increasing the solution frequency, ω_0 , whilst simultaneously increasing the number of subdomains, J .

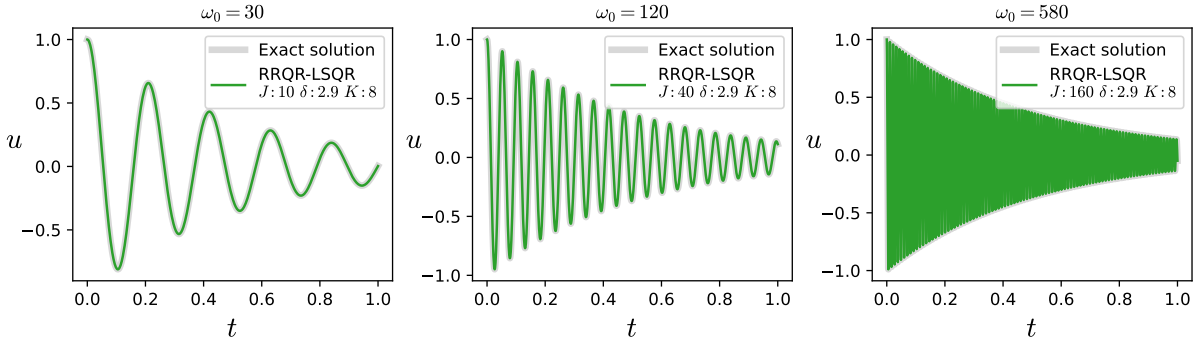


Figure 8: Comparison of exact solution and ELM-FBPINN with RRQR filtering solution for three of the cases in the weak scaling test for the harmonic oscillator in 1D.

Baseline comparison. We compare our baseline ELM-FBPINN with RRQR filtering above to our baseline models. A comparison of the relevant condition numbers, test accuracy and total training time is shown in Table 2. Similar results can be drawn to the harmonic oscillator test above; our model has the highest test accuracy, with the ELM-FBPINN with AS preconditioning and ELM-FBPINNs without preconditioning achieving a slightly lower test accuracy with comparable training times, whilst the FBPINN and PINN baselines perform significantly worse than our model in both accuracy and training time. The convergence curves of all linear solver models are compared in Figure 10. Similar to the harmonic oscillator test, we find that the ELM-FBPINNs without preconditioning take the full 10,000 iteration steps to converge, whilst both our model and the ELM-FBPINN with AS preconditioning converge within a few hundred steps, again showing that preconditioning is essential for enabling the solver to converge quickly. Similar instabilities in the ELM-FBPINN with AS preconditioning at later iteration steps are observed.

Ablation tests. We perform ablation tests on our baseline ELM-FBPINN with RRQR filtering. Specifically, we test its performance using the same ablations studied for the harmonic oscillator problem. The test accuracy and total training time of these models is reported in Table 2. The same conclusions as the harmonic oscillator ablation studies can be drawn, except that increasing the subdomain network depth does not improve accuracy in this case.

Strong scaling tests. We perform a strong scaling test in a similar fashion to the harmonic oscillator test, fixing the problem complexity by keeping $n = 3$, and increasing the model capacity in two different ways; first, by varying K between 4, 8, 16, 32, and 64, and second by fixing $K = 16$, but simultaneously varying S between 8, 16, and 32 and δ between 1.45, 2.9, and 5.8. Plots of the convergence curves and test accuracy versus total training time of the ELM-FBPINN with RRQR filtering, ELM-FBPINN without preconditioning with LSQR, and ELM-FBPINN with AS preconditioning for each scaling test are shown in Figure 11 and Figure 12. For all solvers, we find that the test accuracy improves when increasing K , showing good scaling behavior, similar to the harmonic oscillator problem. However, for the largest K value, the ELM-FBPINN without preconditioning with LSQR is an order

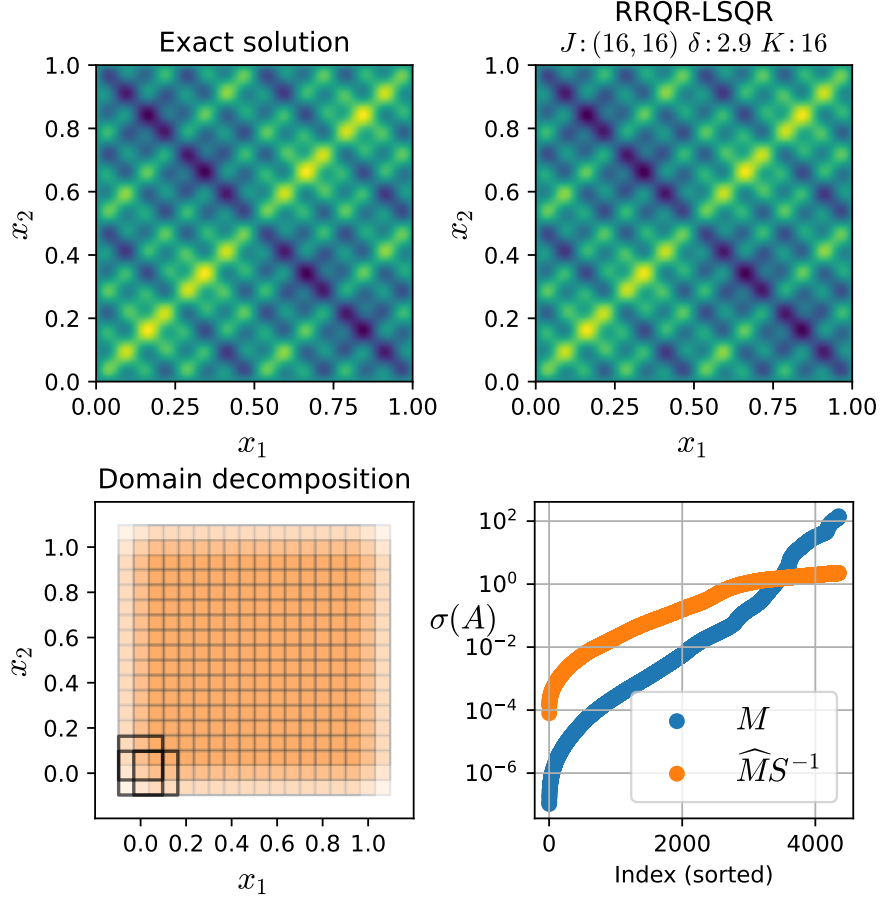


Figure 9: Exact solution (top left), domain decomposition (bottom left), model solution (top right), and SVD spectra of $\mathbf{M}$ and $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ (bottom right) when using the ELM-FBPINN with RRQR filtering to solve the multi-scale Laplacian in 2D.

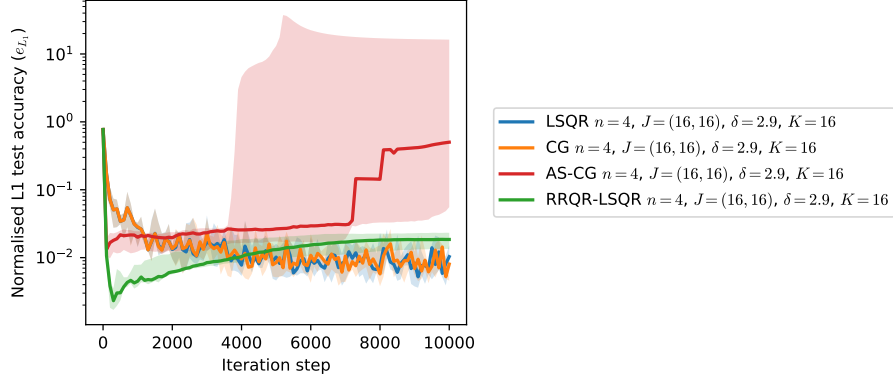


Figure 10: Convergence curves of the ELM-FBPINN with RRQR filtering and our baseline models when solving the multi-scale Laplacian in 2D.

of magnitude less accurate than the other solvers, suggesting that good preconditioning is necessary with higher K values. The ELM-FBPINN with AS preconditioning and our model perform with comparable accuracy, but, similar to the harmonic oscillator problem, the ELM-FBPINN with AS preconditioning is approximately an order of magnitude slower to train, due to the expensive computation of the preconditioner. For all solvers, we find that the test accuracy improves when simultaneously increasing S and δ , however the ELM-FBPINN with AS preconditioning ran out of memory for the last case, and the ELM-FBPINN without preconditioning with LSQR performed similarly to our model, likely because the condition number of $\mathbf{M}$ is relatively low in all cases.

Network	(h, K)	$\kappa(M)$	$\kappa(\widehat{M})$	$\kappa(\widehat{M}S^{-1})$	$\kappa(M^T M)$	$\kappa(A_{AS}^{-1}A)$	Optimiser	$\sigma(\phi_{jk} \text{ drop } \%)$	e_{L_1}	Time (s)
FCN	(2, 128)	N/A	N/A	N/A	N/A	N/A	PINN-Adam	N/A (N/A)	$7.9 \pm 10.8\text{e-}02$	$3.2 \pm 0.0\text{e+}03$
FCN	(1, 16)	N/A	N/A	N/A	N/A	N/A	FBPINN-Adam	N/A (N/A)	$5.0 \pm 2.6\text{e-}03$	$1.5 \pm 0.0\text{e+}03$
ELM	(1, 16)	$1.4\text{e+}09$	N/A	N/A	N/A	N/A	LSQR	N/A (N/A)	$3.9 \pm 0.5\text{e-}03$	$3.1 \pm 0.1\text{e+}01$
ELM	(1, 16)	$1.4\text{e+}09$	N/A	N/A	$8.3\text{e+}18$	N/A	CG	N/A (N/A)	$4.2 \pm 1.1\text{e-}03$	$3.0 \pm 0.1\text{e+}01$
ELM	(1, 16)	$1.4\text{e+}09$	N/A	N/A	$8.3\text{e+}18$	$4.0\text{e+}15$	AS-CG	N/A (N/A)	$1.3 \pm 0.9\text{e-}02$	$1.1 \pm 0.0\text{e+}02$
ELM	(1, 16)	$1.4\text{e+}09$	$1.4\text{e+}09$	$2.8\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-08 (0%)	$1.9 \pm 1.3\text{e-}03$	$3.1 \pm 0.1\text{e+}01$
ELM	(1, 16)	$1.4\text{e+}09$	$1.4\text{e+}09$	$2.8\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-10 (0%)	$1.9 \pm 1.3\text{e-}03$	$3.1 \pm 0.1\text{e+}01$
ELM	(1, 16)	$1.4\text{e+}09$	$9.3\text{e+}08$	$2.2\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-06 (0%)	$1.9 \pm 1.5\text{e-}03$	$3.7 \pm 0.1\text{e+}01$
ELM	(1, 16)	$1.4\text{e+}09$	$3.8\text{e+}06$	$1.1\text{e+}03$	N/A	N/A	RRQR-LSQR	1e-04 (21%)	$2.7 \pm 2.2\text{e-}03$	$2.8 \pm 0.1\text{e+}01$
ELM	(1, 16)	$1.4\text{e+}09$	$6.8\text{e+}03$	$1.1\text{e+}02$	N/A	N/A	RRQR-LSQR	1e-02 (65%)	$1.3 \pm 0.6\text{e-}02$	$1.7 \pm 0.0\text{e+}01$
ELM- σ	(1, 16)	$1.4\text{e+}11$	$7.6\text{e+}10$	$2.4\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-08 (0%)	$2.1 \pm 0.9\text{e-}03$	$3.5 \pm 0.7\text{e+}01$
ELM-sin	(1, 16)	$1.5\text{e+}10$	$1.5\text{e+}10$	$2.2\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-08 (0%)	$2.2 \pm 0.6\text{e-}03$	$2.8 \pm 0.1\text{e+}01$
ELM	(3, 16)	$1.1\text{e+}09$	$1.1\text{e+}09$	$1.7\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-08 (0%)	$2.8 \pm 2.1\text{e-}03$	$4.8 \pm 0.1\text{e+}01$
ELM	(5, 16)	$1.2\text{e+}10$	$1.2\text{e+}10$	$1.9\text{e+}04$	N/A	N/A	RRQR-LSQR	1e-08 (0%)	$2.7 \pm 2.8\text{e-}03$	$6.2 \pm 0.1\text{e+}01$

Table 2: Comparison of test error, total training time, and conditioning numbers for different models solving the multi-scale Laplacian in 2D.

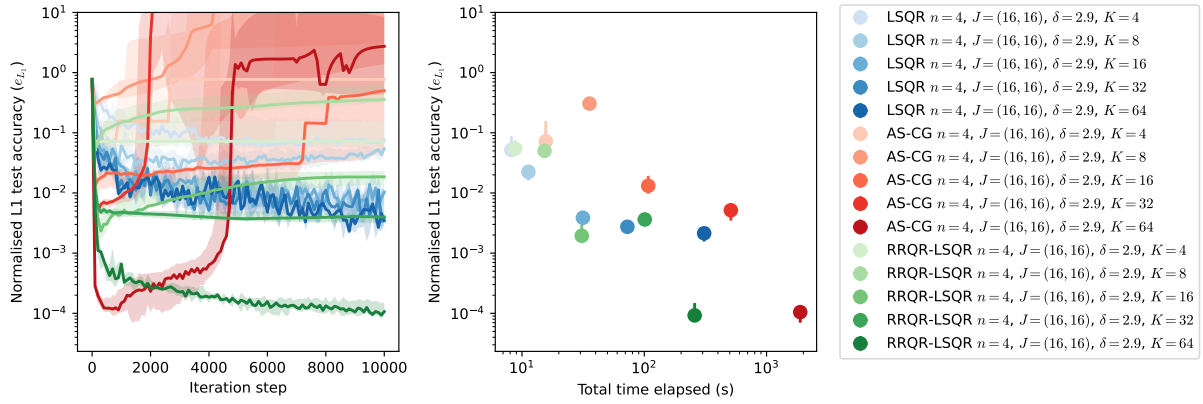


Figure 11: Strong scaling test for the multi-scale Laplacian in 2D, fixing the number of solution components, n , and varying the number of basis functions, K .

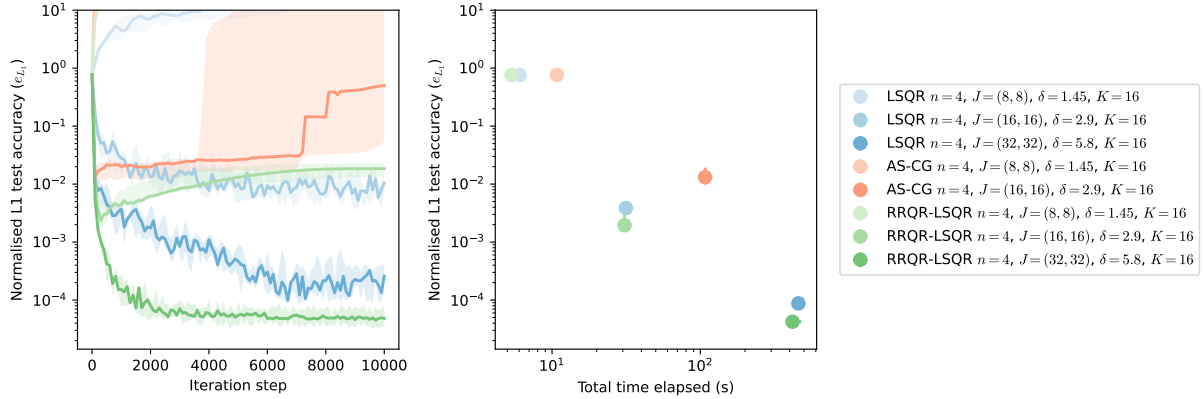


Figure 12: Strong scaling test for the multi-scale Laplacian in 2D, fixing the number of solution components, n , and simultaneously varying the number of subdomains, J , and the overlap between subdomains, δ .

Weak scaling test. We perform a weak scaling test in a similar fashion to the harmonic oscillator test, increasing the problem complexity by varying n between 1, 2, 3, 4, 5, and 6, whilst simultaneously increasing S between 2, 4, 8, 16, 32 and 64. Plots of the convergence curves and test accuracy versus total training time of the ELM-FBPINN with RRQR filtering, ELM-FBPINN without preconditioning with LSQR, and ELM-FBPINN with AS preconditioning for each case are shown in Figure 13, and the exact solution and ELM-FBPINN with RRQR filtering solution are plotted for different n values in Figure 14. We find that all solvers are able to solve all cases to a reasonable accuracy, but undesirably their accuracy reduces as n increases. The final case has $J = 64 \times 64 = 4,096$ subdomains; we postulate that the worsening error is because adding subdomains without increasing their overlap

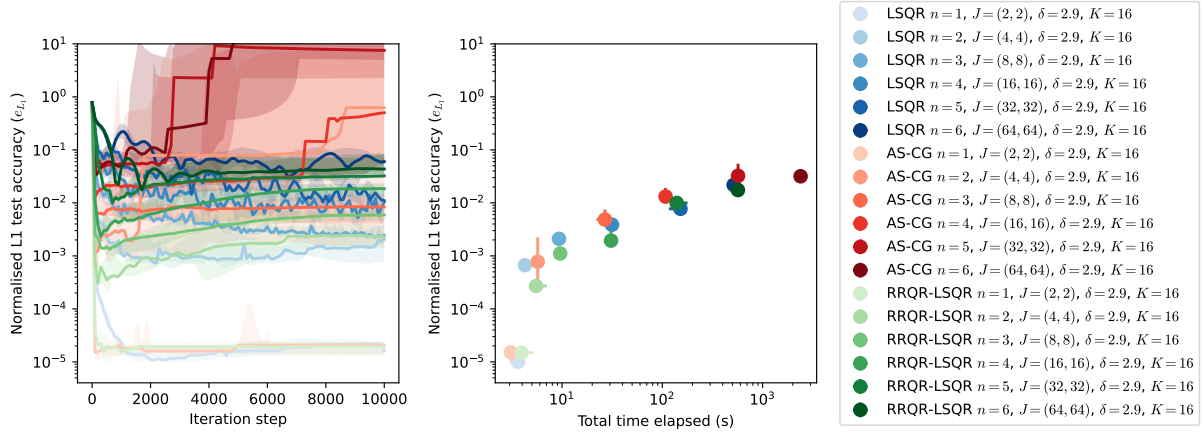


Figure 13: Weak scaling test for the multi-scale Laplacian in 2D, increasing the number of solution components, n , whilst simultaneously increasing the number of subdomains, J .

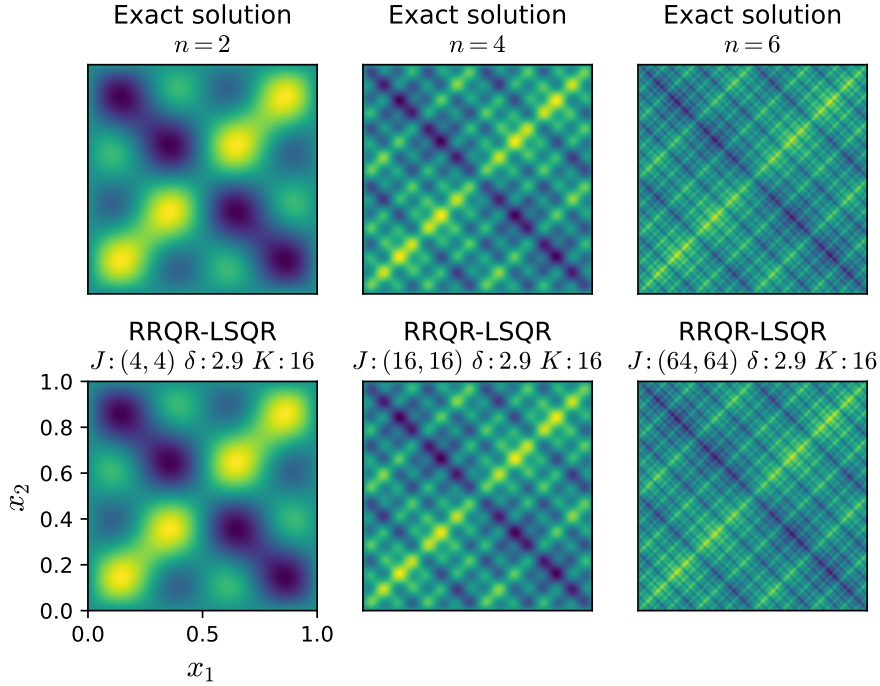


Figure 14: Comparison of exact solution and ELM-FBPINN with RRQR filtering solution for three of the cases in the weak scaling test for the multi-scale Laplacian in 2D.

makes it challenging to communicate between subdomains, potentially making the model harder to train, as studied by [15]. Increasing the overlap reduces sparsity in $\mathbf{M}$, significantly increasing memory and training time, such that we were unable to increase the overlap significantly without running out of memory. In future work, a more promising direction may be to instead aid communication between subdomains by incorporating multiple levels of domain decompositions, as studied by [15].

4.4.3. Time-varying wave equation in $(2+1)D$

Baseline model. Finally, we study the $(2+1)D$ wave equation problem. We train a baseline ELM-FBPINN with RRQR filtering, studying the problem with $\lambda = 0.2$, using $S = 8$, $\delta = 2.9$, $K = 8$, a subdomain network depth of $h = 1$, tanh activation function, $\sigma = 1 \times 10^{-8}$, and $4 \times S = 32$ collocation points along each input dimension for testing. A comparison of the solution from FD simulation and our model solution is shown in Figure 15. We find that the model is able to predict the solution with reasonable accuracy, with a normalized L1 test accuracy of $2.2 \pm 0.4 \times 10^{-2}$.

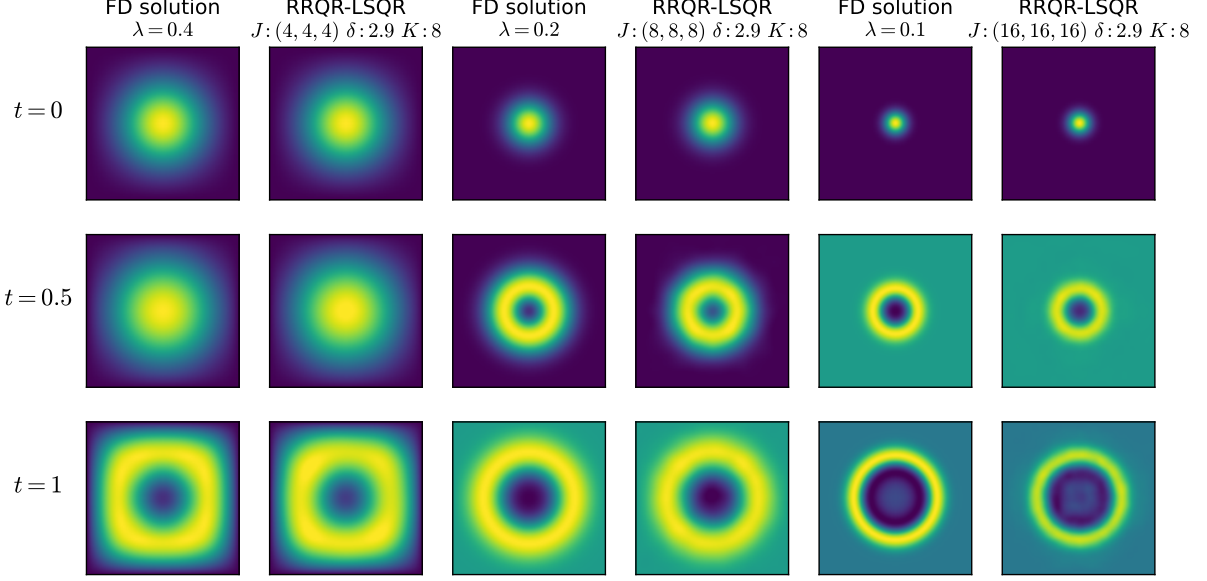


Figure 15: Comparison of exact solution and ELM-FBPINN with RRQR filtering solution for three of the cases in the weak scaling test for the time-varying wave equation in (2+1)D.

Similarly to the multi-scale Laplacian problem, we find that no columns are dropped during σ -RRQR filtering, likely because the condition number of $\mathbf{M}$ is already relatively low at 5.1×10^6 . After preconditioning the condition number of $\widehat{\mathbf{M}}\mathbf{S}^{-1}$ further drops to 2.4×10^2 . The convergence of the LSQR solver is shown in Figure 16. We find that the model converges within a few tens of steps, whilst the total time taken to carry out the full 10,000 iterations is $1.0 \pm 0.0 \times 10^2$ s.

Baseline comparison. We compare our baseline ELM-FBPINN with RRQR filtering above to our baseline models. A comparison of the relevant condition numbers, test accuracy and total training time is shown in Table 3. In this case, all baseline models achieve a similar level of accuracy, but the training time for the PINN and FBPINN models is two orders of magnitude larger than the linear solver models. The convergence curves of all linear solver models are compared in Figure 16. Similar to the harmonic oscillator and multi-scale Laplacian problem, we find that the ELM-FBPINNs without preconditioning take the full 10,000 iteration steps to converge, whilst both our model and the ELM-FBPINN with AS preconditioning converge within a few tens of steps.

Network	(h, K)	$\kappa(\mathbf{M})$	$\kappa(\widehat{\mathbf{M}})$	$\kappa(\widehat{\mathbf{M}}\mathbf{S}^{-1})$	$\kappa(\mathbf{M}^T \mathbf{M})$	$\kappa(\mathbf{A}_{AS}^{-1} \mathbf{A})$	Optimiser	σ (ϕ_{jk} drop %)	e_{L_1}	Time (s)
FCN	(2, 128)	N/A	N/A	N/A	N/A	N/A	PINN-Adam	N/A (N/A)	$2.9 \pm 1.8e-02$	$1.1 \pm 0.0e+04$
FCN	(1, 8)	N/A	N/A	N/A	N/A	N/A	FBPINN-Adam	N/A (N/A)	$1.9 \pm 0.9e-02$	$1.0 \pm 0.0e+04$
ELM	(1, 8)	$5.1e+06$	N/A	N/A	N/A	N/A	LSQR	N/A (N/A)	$2.4 \pm 0.3e-02$	$1.2 \pm 0.1e+02$
ELM	(1, 8)	$5.1e+06$	N/A	N/A	$2.6e+13$	N/A	CG	N/A (N/A)	$2.4 \pm 0.3e-02$	$10.0 \pm 1.3e+01$
ELM	(1, 8)	$5.1e+06$	N/A	N/A	$2.6e+13$	$3.8e+08$	AS-CG	N/A (N/A)	$2.3 \pm 0.4e-02$	$6.2 \pm 0.1e+02$
ELM	(1, 8)	$5.1e+06$	$5.1e+06$	$2.4e+02$	N/A	N/A	RRQR-LSQR	$1e-08$ (0%)	$2.2 \pm 0.4e-02$	$1.0 \pm 0.0e+02$
ELM	(1, 8)	$5.1e+06$	$5.1e+06$	$2.4e+02$	N/A	N/A	RRQR-LSQR	$1e-10$ (0%)	$2.2 \pm 0.4e-02$	$8.5 \pm 0.1e+01$
ELM	(1, 8)	$5.1e+06$	$5.1e+06$	$2.4e+02$	N/A	N/A	RRQR-LSQR	$1e-06$ (0%)	$2.2 \pm 0.4e-02$	$1.3 \pm 0.3e+02$
ELM	(1, 8)	$5.1e+06$	$8.7e+05$	$2.4e+02$	N/A	N/A	RRQR-LSQR	$1e-04$ (0%)	$2.2 \pm 0.4e-02$	$1.3 \pm 0.3e+02$
ELM	(1, 8)	$5.1e+06$	$1.1e+04$	$8.2e+01$	N/A	N/A	RRQR-LSQR	$1e-02$ (38%)	$4.7 \pm 1.1e-02$	$6.8 \pm 0.1e+01$
ELM- σ	(1, 8)	$9.9e+07$	$9.9e+07$	$2.9e+02$	N/A	N/A	RRQR-LSQR	$1e-08$ (0%)	$2.3 \pm 0.5e-02$	$8.4 \pm 0.2e+01$
ELM-sin	(1, 8)	$8.9e+06$	$8.9e+06$	$2.9e+02$	N/A	N/A	RRQR-LSQR	$1e-08$ (0%)	$2.3 \pm 0.5e-02$	$9.3 \pm 0.1e+01$
ELM	(3, 8)	$9.5e+06$	$9.5e+06$	$2.5e+02$	N/A	N/A	RRQR-LSQR	$1e-08$ (0%)	$2.2 \pm 0.2e-02$	$1.6 \pm 0.1e+02$
ELM	(5, 8)	$1.6e+08$	$1.6e+08$	$2.3e+02$	N/A	N/A	RRQR-LSQR	$1e-08$ (0%)	$2.4 \pm 0.8e-02$	$1.4 \pm 0.0e+02$

Table 3: Comparison of test error, total training time, and conditioning numbers for different models solving the time-varying wave equation in (2+1)D.

Ablation tests. We perform the same suite of ablation tests on our model as above. The test accuracy and total training time of these models is reported in Table 3. The same conclusions as the multi-scale Laplacian problem can be drawn.

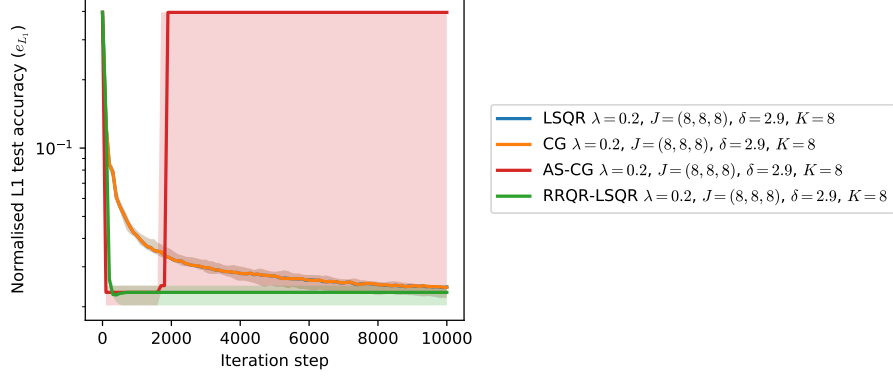


Figure 16: Convergence curves of the ELM-FBPINN with RRQR filtering and our baseline models when solving the time-varying wave equation in (2+1)D.

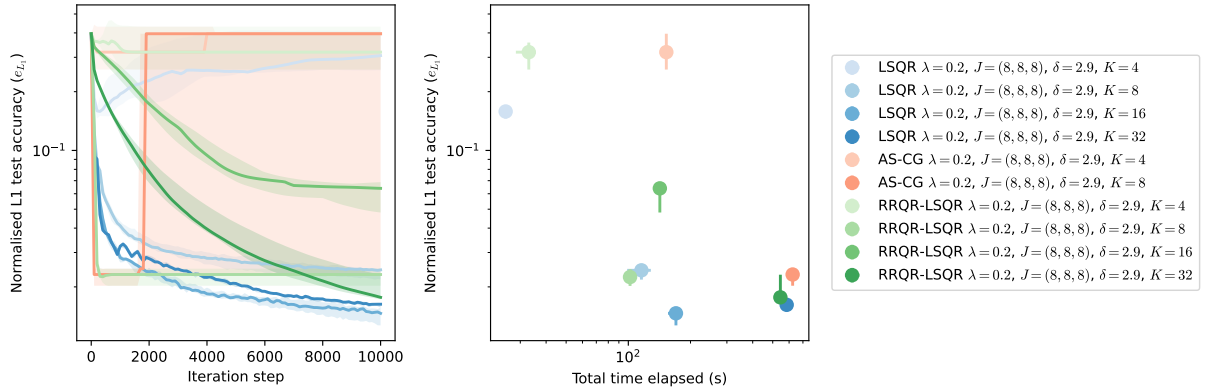


Figure 17: Strong scaling test for the time-varying wave equation in (2+1)D, fixing the solution wavelength, λ , and varying the number of basis functions, K .

Strong scaling tests. We perform a strong scaling test, fixing the problem complexity by keeping $\lambda = 0.2$, and increasing the model capacity first by varying K between 4, 8, 16, and 32, and second by fixing $K = 8$, but simultaneously varying S between 4, 8, and 16 and δ between 1.45, 2.9, and 5.8. Plots of the convergence curves and test accuracy versus total training time of the ELM-FBPINN with RRQR filtering, ELM-FBPINN without preconditioning with LSQR, and ELM-FBPINN with AS preconditioning for each scaling test are shown in Figure 17 and Figure 18. For all solvers, we find that the test accuracy improves when increasing K , showing good scaling behavior. However, the ELM-FBPINN with AS preconditioning is approximately an order of magnitude slower to train, and runs out of memory for the last case. Furthermore, the convergence rate of our model significantly worsens as K increases. When using a regular overlapping rectangular domain decomposition, the maximum number of overlapping models increases exponentially with the number of input dimensions d , so we postulate that our block-wise preconditioning may not be as effective with high d because it neglects the overlapping contributions between subdomains in $\mathbf{M}$. Improving the performance of our preconditioner in higher dimensions will be the subject of future work. For all solvers, similar to the harmonic oscillator and multi-scale Laplacian problem, we find that the test accuracy improves when simultaneously increasing S and δ .

Weak scaling test. Finally, we perform a weak scaling test, increasing the problem complexity by varying λ between 0.4, 0.2 and 0.1, whilst simultaneously increasing S between 4, 8, and 16. Plots of the convergence curves and test accuracy versus total training time of the ELM-FBPINN with RRQR filtering, ELM-FBPINN without preconditioning with LSQR, and ELM-FBPINN with AS preconditioning for each case are shown in Figure 19, and the exact solution and ELM-FBPINN with RRQR filtering solution are plotted for different λ values in Figure 15. Similarly to the multi-scale Laplacian problem, we find that all solvers are able to solve all cases to a reasonable accuracy, but with their accuracy reducing as λ decreases. As suggested above, improving communication between subdomains by adding multiple levels of domain decompositions may improve performance further.

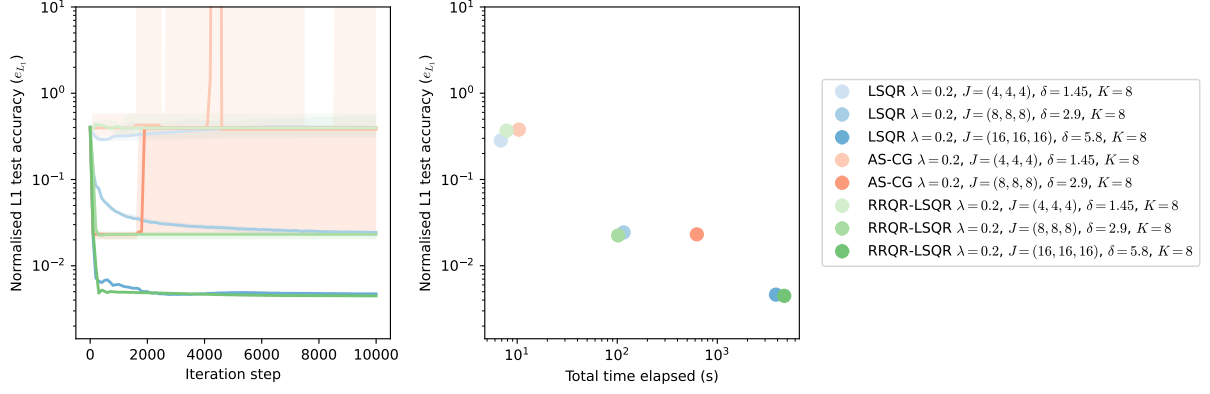


Figure 18: Strong scaling test for the time-varying wave equation in (2+1)D, fixing the solution wavelength, λ , and simultaneously varying the number of subdomains, J , and the overlap between subdomains, δ .

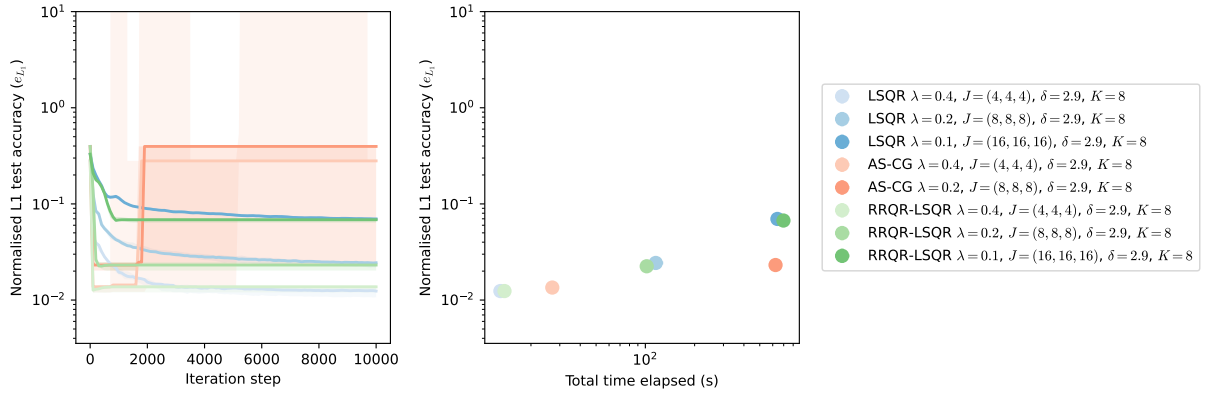


Figure 19: Weak scaling test for the time-varying wave equation in (2+1)D, increasing the solution wavelength, λ , whilst simultaneously increasing the number of subdomains, J .

4.5. General observations and future work

Across all experiments, we find that our σ -RRQR filtering and preconditioning strategy significantly improves both the test accuracy and convergence rate of ELM-FBPINNs compared to our baseline models. Our filtering and preconditioning always improves the conditioning of the ELM-FBPINN least-squares system, enabling faster convergence rates, with filtering having the added benefit of reducing the size of $\mathbf{M}$ and therefore computational time of the solver. Compared to PINNs and FBPINNs, our solver is multiple orders of magnitude faster, because it does not need to rely on nonlinear gradient descent. Compared to ELM-FBPINNs without preconditioning, our solver generally has a much faster rate of convergence because of preconditioning. Compared to ELM-FBPINN with AS preconditioning, our solver requires significantly less time and memory to compute the preconditioner, and exhibits more stable convergence curves, possibly because we drop highly numerically dependent columns and directly solve the least-squares system instead of its normal equations.

We identified multiple directions for future work and outstanding limitations of our method, including improving performance for larger numbers of subdomains, perhaps by extending our framework to multilevel domain decompositions, as proposed by [15], and investigating improvements for higher numbers of input dimensions, perhaps by incorporating overlapping information into our preconditioner. It is interesting to note that whilst our preconditioning always improves the conditioning of the system, it does not always improve the final test error achieved by the solver. Further work could be carried out to understand this behavior more. Future work could also include further accelerating our solvers, which are currently implemented in SciPy, by implementing them on the GPU, and building further theoretical understanding on the convergence rate of ELM-FBPINNs.

Appendix A. Probabilistic refinements via random matrix theory

While the main analysis in Section 3 is deterministic, it is often instructive to understand the *typical-case* behaviour of the preconditioned system when the features are randomized. In practice, the subdomain matrices obtained after local RRQR filtering tend to have nearly orthogonal columns and exhibit statistical independence across subdomains, a consequence of the random feature construction. To formalize this intuition, we model the filtered subdomain blocks as random semi-orthogonal matrices.

Appendix A.1. Random orthogonal matrices

Specifically, we model the subdomain-restricted matrices produced by RRQR as random semi-orthogonal matrices, reflecting their approximate local orthogonality and statistical independence due to randomized features. This abstraction allows us to analytically study how the conditioning of the global system changes as a function of overlap size. In this section, we collect relevant results on random orthogonal and semi-orthogonal matrices that will be used later in this section. We will denote random matrices with calligraphic letters (i.e. $\mathcal{Q}$ instead of Q) to distinguish them from ordinary matrices.

We start by recalling a definition of a natural uniform distribution on the group $O_n(\mathbb{R})$ [27]:

Definition Appendix A.1 (Random orthogonal matrix). *Let $O_n(\mathbb{R})$ denote the group of $n \times n$ orthogonal matrices over $\mathbb{R}$. There exists a unique probability measure μ on $O_n(\mathbb{R})$ such that if $Q \sim \mu$ (i.e. is distributed according to μ), then for all $U, V \in O_n(\mathbb{R})$, the matrix UQV has the same distribution as Q . A matrix drawn from this distribution is called a random orthogonal matrix distributed according to the Haar measure on $O_n(\mathbb{R})$.*

The following useful and constructive characterization of such matrices is also given in [27].

Proposition Appendix A.2 (Characterisation of random orthogonal matrices). *Let X be an $n \times n$ matrix with entries independently distributed according to a standard normal distribution. Let $X = QR$ be the QR-factorization, where all diagonal elements of R are normalized to be positive. Then Q is a random orthogonal matrix according to the Haar measure on $O_n(\mathbb{R})$.*

From this it follows that for each column $Q^{(i)}$ separately, one has

$$Q_j^{(i)} = \frac{X_j}{\sqrt{\sum_{k=1}^n X_k^2}}, \quad (\text{A.1})$$

with the X_j independent standard normal random variables. This is certainly true for the first column, as the first step of Gram-Schmidt simply normalizes the first column of X . By the invariance of Q under column permutations (which follows from definition Appendix A.1), it then follows for the other columns as well.

Definition Appendix A.3 (Random semi-orthogonal matrix). *Let Q be a random orthogonal matrix of size $m \times m$, and let $\mathcal{A}$ be the $m \times k$ matrix obtained by selecting any $k \leq m$ columns of Q . Then $\mathcal{A}$ is called a random semi-orthogonal matrix.*

By Theorem Appendix A.1, the distribution of $\mathcal{A}$ does not depend on the particular choice of columns of Q . These matrices are also called k -frames in the random matrix literature [28]. We call them random semi-orthogonal here to stress the fact that $\mathcal{A}^T \mathcal{A} = I$.

The following result, i.e. that the product of two independent Haar-distributed orthogonal matrices is itself a Haar-distributed is not usually presented as standalone theorem in the relevant literature [29, 30] but rather as a consequence of the invariance properties of the Haar measure. It is considered standard and foundational in the theory of probability on groups. In order to make the presentation self consistent and accessible in the current context we state this here and give a short proof for random orthogonal matrices.

Proposition Appendix A.4 (Invariance under multiplication). *Let $\mathcal{P}, \mathcal{Q} \in O_n(\mathbb{R})$ be independent random orthogonal matrices. Then their product $\mathcal{Z} = \mathcal{P}\mathcal{Q}$ is also a random orthogonal matrix. In particular, $\mathcal{Z}$ has the same distribution as $\mathcal{P}$ and $\mathcal{Q}$.*

Proof. The key property of the Haar measure is its invariance under both left and right multiplication by fixed orthogonal matrices. Concretely, if Q is Haar-distributed and $P \in O_n(\mathbb{R})$ is fixed, then the product PQ has the same distribution as Q . This invariance implies that multiplying a Haar-distributed matrix by a fixed orthogonal matrix does not alter its distribution.

Now, let $\mathcal{P}$ and Q be independent Haar-distributed orthogonal matrices, and define $\mathcal{Z} = \mathcal{P}Q$. We want to show that $\mathcal{Z}$ is also Haar-distributed.

Let $H \subset O_n(\mathbb{R})$ be any measurable subset. Conditioning on a fixed realization $\mathcal{P} = P$, we have:

$$\mathbb{P}(\mathcal{Z} \in H \mid \mathcal{P} = P) = \mathbb{P}(PQ \in H) = \mathbb{P}(Q \in P^{-1}H),$$

where $P^{-1}H = \{Q \in O_n \mid PQ \in H\}$. Since Q is Haar-distributed and P is fixed, we have

$$\mathbb{P}(Q \in P^{-1}H) = \mathbb{P}(Q \in P^T H) = \mathbb{P}(Q \in H),$$

by left invariance of the Haar measure.

Integrating over all possible values of $\mathcal{P}$, we obtain:

$$\begin{aligned} \mathbb{P}(\mathcal{Z} \in H) &= \int_{O_n} \mathbb{P}(\mathcal{Z} \in H \mid \mathcal{P} = P) d\mu(P) = \int_{O_n} \mathbb{P}(Q \in H) d\mu(P) \\ &= \mathbb{P}(Q \in H) \int_{O_n} d\mu(P) = \mathbb{P}(Q \in H). \end{aligned}$$

since μ is a probability measure. Therefore, $\mathcal{Z}$ is Haar-distributed. $\square$

Remark Appendix A.5 (Numerical intuition). *This result formalizes an intuitive property: if we multiply two orthogonal matrices, each randomly sampled in a systematic way (e.g., using the QR decomposition of Gaussian matrices), the result is still randomly distributed in the same way in the space of orthogonal matrices. This closure under multiplication is a feature of compact groups like $O_n(\mathbb{R})$, and ensures that the "orthogonal randomness" is preserved under composition, an important property in randomized algorithms and preconditioner analysis. We say that the statistical properties of such matrices are preserved under orthogonal transformations and compositions and their use in iterative solvers or algorithms does not introduce additional bias or artifacts.*

For our application, we are interested in the operator norm of the product of blocks of orthogonal matrices. A precise investigation of this quantity is beyond the scope of this paper. Instead, we give a bound on the expected Frobenius norm, which is easier to handle.

Proposition Appendix A.6 (Expected Frobenius norm of product). *Let $\mathcal{P}, Q \in O_n(\mathbb{R})$ be independent random orthogonal matrices drawn from the Haar measure. Let $\mathcal{P}_1, Q_1$ be $\ell \times K$ blocks of $\mathcal{P}$ and Q respectively. Then we have*

$$\mathbb{E} \|Q_1^T \mathcal{P}_1\|_F \leq \frac{K \sqrt{\ell}}{n}.$$

Proof. Let $\mathcal{A} = Q_1^T \mathcal{P}_1$, with entries a_{ij} for $i, j \in \{1, \dots, K\}$. By A.1, the columns of Q_1 have entries of the form

$$Q_k^{(m)} = \frac{X_k}{\sqrt{\sum_{k=1}^n X_k^2}},$$

with the X_k independent standard normal random variables. The columns of $\mathcal{P}_1$ have the same form. Hence an entry a_{ij} of $\mathcal{A}$ is of the form

$$a_{ij} = \sum_{k=1}^{\ell} \frac{X_k Y_k}{\sqrt{\sum_{k=1}^n X_k^2} \sqrt{\sum_{k=1}^n Y_k^2}},$$

with the X_k, Y_k independent standard normal. Taking the square and expectation of this expression yields

$$\mathbb{E} a_{ij}^2 = \sum_{k=1}^{\ell} \left(\mathbb{E} \frac{X_k^2}{\sum_{k=1}^n X_k^2} \mathbb{E} \frac{Y_k^2}{\sum_{k=1}^n Y_k^2} \right) = \sum_{k=1}^{\ell} \frac{1}{n^2} = \frac{\ell}{n^2}.$$

Note that we can eliminate the cross terms in the above square, since they have expectation 0. It follows that

$$\mathbb{E}\|\mathcal{A}\|_F = \mathbb{E} \sqrt{\sum_{i,j \leq K} a_{ij}^2} \leq \sqrt{\mathbb{E} \sum_{i,j \leq K} a_{ij}^2} = \sqrt{\sum_{i,j \leq K} \mathbb{E} a_{ij}^2} = \frac{K\sqrt{\ell}}{n},$$

where we have used Jensen's inequality and the fact that taking the square root is a concave function. $\square$

Remark Appendix A.7. *It is easily calculated that*

$$\mathbb{E}\|\mathcal{P}_1\|_F = \mathbb{E}\|\mathcal{Q}_1\|_F = \sqrt{\frac{K\ell}{n}}.$$

Applying the submultiplicativity of the Frobenius norm and using the independence of $\mathcal{P}_1$ and $\mathcal{Q}_1$ would give

$$\mathbb{E}\|\mathcal{Q}_1^T \mathcal{P}_1\|_F \leq \frac{K\ell}{n}.$$

So the above derived bound is slightly tighter than this.

Finally we cite the following bound from [28], which uses far more sophisticated methods to estimate the spectral norm of a submatrix directly.

Proposition Appendix A.8 (Expected spectral norm of a principal block [28]). *Let $\mathcal{P}_1 \in \mathbb{R}^{\ell \times K}$ be the top-left block of a Haar-distributed orthogonal matrix $\mathcal{P} \in O_n(\mathbb{R})$. Then*

$$\mathbb{E}\|\mathcal{P}_1\| \leq \left(1 + \frac{K}{2n}\right) \frac{\sqrt{K} + \sqrt{\ell}}{\sqrt{n}}.$$

Proof. Although rigorous proof of this result is more involved we give here the main ideas. To estimate the expected norm of the principal block $\mathcal{P}_1$ of a Haar-distributed orthogonal matrix, we leverage the rotational invariance of the Haar measure, which implies that any fixed $\ell \times K$ block has the same distribution. The behavior of $\mathcal{P}_1$ is then compared to that of a normalized Gaussian matrix with i.i.d. $\mathcal{N}(0, 1)$ entries, whose expected operator norm is known to be bounded by $\sqrt{K} + \sqrt{\ell}$. This yields a baseline estimate of $(\sqrt{K} + \sqrt{\ell})/\sqrt{n}$. Finally, Tropp [28] refines this comparison by accounting for the difference between the true Haar marginal and the Gaussian approximation, introducing a correction factor $(1 + \frac{K}{2n})$ that captures the residual dependence structure in $\mathcal{P}_1$. $\square$

Corollary Appendix A.9. *Let $\mathcal{P}_1$ and $\mathcal{Q}_1$ be as in proposition Appendix A.6. Let $\varepsilon_{\ell,K,n}$ denote the quantity*

$$\varepsilon_{\ell,K,n} = \left(1 + \frac{K}{2n}\right) \frac{\sqrt{K} + \sqrt{\ell}}{\sqrt{n}}.$$

Then we have

$$\mathbb{E}\|\mathcal{Q}_1^T \mathcal{P}_1\|_2 \leq \varepsilon_{\ell,K,n}^2.$$

Proof. This follows directly from proposition Appendix A.8 and the submultiplicativity of the spectral norm. $\square$

Remark Appendix A.10. *Applying the methods of [28] directly to $\mathcal{Q}_1^T \mathcal{P}_1$ would likely lead to a tighter estimate than using the submultiplicativity of the spectral norm. However, this is beyond the scope of this paper.*

Appendix A.2. Estimating block interactions via random matrix theory

To carry out a probabilistic analysis, we focus on the interaction matrices $\mathbf{A}_j = \mathbf{Q}_{j+1,j}^T \mathbf{Q}_{j,j} \in \mathbb{R}^{K \times K}$ arising from the overlapping blocks $\mathbf{Q}_{j+1,j}, \mathbf{Q}_{j,j} \in \mathbb{R}^{\ell \times K}$. When the features within each subdomain are generated by a randomized construction, it is natural to model each block as a realisation of a random matrix $\mathcal{Q}$ representing the top ℓ rows of the first K columns of an $n \times n$ Haar-distributed orthogonal matrix $\mathcal{P} \in O_n$. As in the previous paragraph, let $\varepsilon_{\ell,K,n}$ denote

$$\varepsilon_{\ell,K,n} = \left(1 + \frac{K}{2n}\right) \frac{\sqrt{K} + \sqrt{\ell}}{\sqrt{n}}.$$

Applying Appendix A.9 to $\mathbf{A}_j$ yields

$$\mathbb{E}\|\mathbf{A}_j\|_2 \leq \varepsilon_{\ell,K,n}^2. \tag{A.2}$$

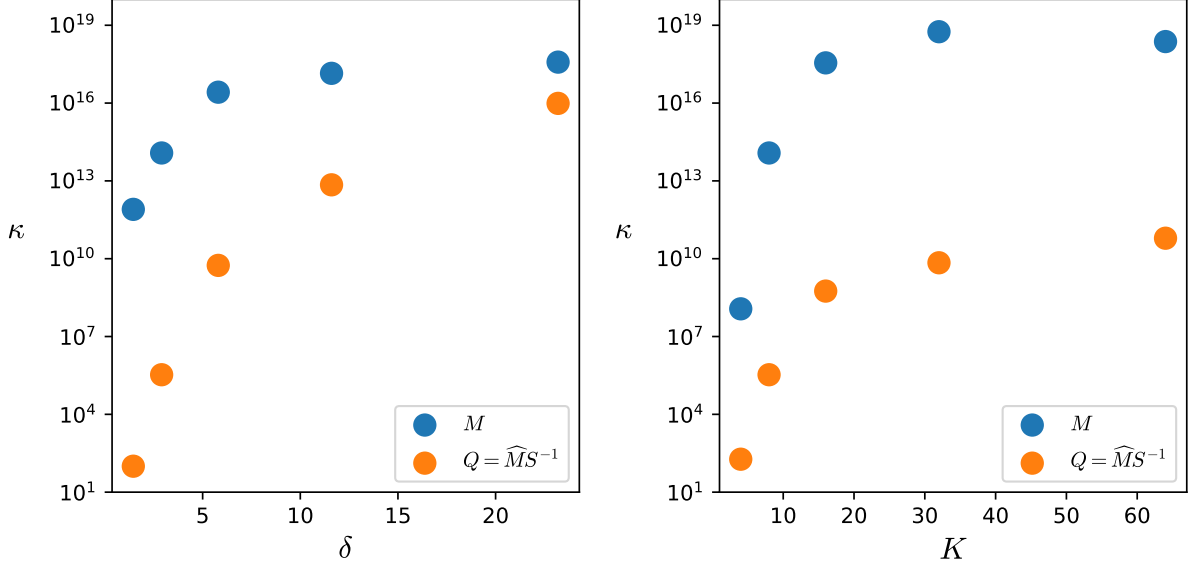


Figure A.20: Plots showing how the condition number of $\mathbf{M}$ and $\mathbf{Q}$ vary with K and δ when solving the harmonic oscillator in 1D with our baseline ELM-FBPINN with RRQR filtering described in Section 4.4.1.

From α to $\kappa(\mathbf{Q})$. The deterministic bound in Theorem 3.2 holds with $\alpha = \max_j \|\mathbf{A}_j\|_2$. Approximating α by its typical RMT value $\varepsilon_{\ell,K,n}$, and recalling that all $\mathbf{A}_j$ are identically distributed and independent, we arrive at the probabilistic estimate

$$\mathbb{E}[\kappa(\mathbf{Q})] \lesssim \left(\frac{1 + 2\varepsilon_{\ell,K,n}^2}{1 - 2\varepsilon_{\ell,K,n}^2} \right)^{1/2}, \quad \text{valid when } 2\varepsilon_{\ell,K,n} < 1. \quad (\text{A.3})$$

Remark Appendix A.11. The quantity $\varepsilon_{\ell,K,n}$ in Equation (A.2) encodes the relative feature complexity K/n and the relative overlap size ℓ/n . Small K and small ℓ produce small $\varepsilon_{\ell,K,n}$ and hence a well-conditioned $\mathbf{Q}$. Conversely, if either K/n or $\sqrt{\ell/n}$ is large, then $\varepsilon_{\ell,K,n}^2$ approaches 1/2 and the bound (A.3) is no longer effective. From (A.3) we can distinguish two regimes:

- **Small-overlap regime:** If $\ell \ll n$ and $K \ll n$, then $\varepsilon_{\ell,K,n} \rightarrow 0$ and

$$\mathbb{E}[\kappa(\mathbf{Q})] \approx 1 + 2\varepsilon_{\ell,K,n}^2 + \mathcal{O}(\varepsilon_{\ell,K,n}^4),$$

indicating very mild conditioning loss.

- **Large-overlap regime:** As ℓ/n increases, $\varepsilon_{\ell,K,n}$ grows and the denominator $1 - 2\varepsilon_{\ell,K,n}^2$ in (A.3) shrinks. When $\varepsilon_{\ell,K,n}^2 \rightarrow 1/2$, the bound formally blows up and beyond that limit i.e. for ℓ a significant fraction of n , the estimate is no longer applicable.

Numerical validation of bound. Equation (A.2) and Equation (A.3) provide a weak upper bound on $\kappa(\mathbf{Q})$. Specifically, the bound increases with K , the number of basis functions in the model, and ℓ , the number of overlapping rows between each subdomain. We compare this result to how $\kappa(\mathbf{Q})$ changes numerically when increasing K and δ (which analogously to ℓ controls the overlap between subdomains, as defined in Equation (19)) when solving the harmonic oscillator in 1D studied in Section 4.4.1. We define a ELM-FBPINN with RRQR filtering using the same baseline hyperparameters described in Section 4.4.1, except that we fix $\sigma = 0$ and the number of interior collocation points to be 2000. Figure A.20 shows how $\kappa(\mathbf{M})$ and $\kappa(\mathbf{Q})$ vary when changing K and δ . We see the same behavior expected by the bound; both condition numbers increase significantly with increasing K and δ .

Appendix B. Finite difference modeling for time-varying wave equation in (2+1)D

We use finite difference modeling as the ground truth solution when studying the (2+1)D wave equation problem in Section 4.4.3. We use the SEISMIC_CPML library [31] which performs staggered-grid second order finite difference

modeling of the time-dependent 2D acoustic wave equation with Dirichlet boundary conditions. For ease of use, we re-implemented the original Fortran code in Python. The simulation is initialized by sampling the initial wavefield and initial wavefield derivative as defined in Section 4.1.3 on a regular grid. A high density of grid points ($\sim 5\times$ spatial Nyquist frequency) is used to ensure that the simulation is high-fidelity.

References

- [1] J. Chen, X. Chi, W. E, Z. Yang, Bridging traditional and machine learning-based algorithms for solving pdes: the random feature method, *J Mach Learn* 1 (2022) 268–98.
- [2] S. Anderson, V. Dolean, B. Moseley, J. Pestana, ELM-FBPINN: efficient finite-basis physics-informed neural networks, *arXiv preprint arXiv:2409.01949* (2024).
- [3] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Transactions on Neural Networks* 9 (5) (1998) 987–1000. *arXiv:9705023*, doi:10.1109/72.712178.
- [4] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.
URL <https://doi.org/10.1016/j.jcp.2018.10.045>
- [5] M. G. Dissanayake, N. Phan-Thien, Neural-network-based approximations for solving partial differential equations, *Communications in Numerical Methods in Engineering* 10 (3) (1994) 195–201.
- [6] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of neural networks, in: 36th International Conference on Machine Learning, ICML 2019, Vol. 2019-June, International Machine Learning Society (IMLS), 2019, pp. 9230–9239. *arXiv:1806.08734*.
URL <http://arxiv.org/abs/1806.08734>
- [7] B. Moseley, A. Markham, T. Nissen-Meyer, Finite basis physics-informed neural networks (FBPINNs): a scalable domain decomposition approach for solving differential equations, *Advances in Computational Mathematics* 2023 49:4 49 (4) (2023) 1–39. doi:10.1007/S10444-023-10065-9.
URL <https://link.springer.com/article/10.1007/s10444-023-10065-9>
- [8] S. Wang, H. Wang, P. Perdikaris, On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks, *Computer Methods in Applied Mechanics and Engineering* 384 (2021) 113938. *arXiv:2012.10047*, doi:10.1016/j.cma.2021.113938.
- [9] B. Moseley, Physics-informed machine learning: from concepts to real-world applications, Ph.D. thesis, University of Oxford (2022). doi:10.13039/501100000266.
- [10] A. D. Jagtap, E. Kharazmi, G. E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, *Computer Methods in Applied Mechanics and Engineering* 365 (2020) 113028.
- [11] A. D. Jagtap, G. E. Karniadakis, Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, *Communications in Computational Physics* 28 (5) (2020).
- [12] E. Kharazmi, Z. Zhang, G. E. Karniadakis, hp-VPINNs: Variational physics-informed neural networks with domain decomposition, *Computer Methods in Applied Mechanics and Engineering* 374 (2021) 113547.
- [13] W. Li, X. Xiang, Y. Xu, Deep domain decomposition method: Elliptic problems, in: *Mathematical and Scientific Machine Learning*, PMLR, 2020, pp. 269–286.
- [14] V. Dolean, A. Heinlein, S. Mishra, B. Moseley, Finite basis physics-informed neural networks as a Schwarz domain decomposition method, in: *International Conference on Domain Decomposition Methods*, Springer, 2022, pp. 165–172.
- [15] V. Dolean, A. Heinlein, S. Mishra, B. Moseley, Multilevel domain decomposition-based architectures for physics-informed neural networks, *Computer Methods in Applied Mechanics and Engineering* 429 (2024) 117116.
- [16] G. B. Huang, Q. Y. Zhu, C. K. Siew, Extreme learning machine: Theory and applications, *Neurocomputing* 70 (1-3) (2006) 489–501. doi:10.1016/j.neucom.2005.12.126.
- [17] Y.-H. Pao, Y. Takefujii, Functional-link net computing: theory, system architecture, and functionalities, *Computer* 25 (5) (1992) 76–79.
- [18] M. Lukoševičius, H. Jaeger, Reservoir computing approaches to recurrent neural network training, *Computer Science Review* 3 (3) (2009) 127–149. doi:<https://doi.org/10.1016/j.cosrev.2009.03.005>.
URL <https://www.sciencedirect.com/science/article/pii/S1574013709000173>
- [19] C. Datar, T. Kapoor, A. Chandra, Q. Sun, I. Burak, E. L. Bolager, A. Veselovska, M. Fornasier, F. Dietrich, Solving partial differential equations with sampled neural networks (2024). *arXiv:2405.20836*.
URL <https://arxiv.org/abs/2405.20836>
- [20] Y. Shang, A. Heinlein, S. Mishra, F. Wang, Overlapping schwarz preconditioners for randomized neural networks with domain decomposition, *Computer Methods in Applied Mechanics and Engineering* 442 (1) (Jul. 2025). doi:10.1016/j.cma.2025.118011.
- [21] T. F. Chan, Rank revealing qr factorizations, *Linear Algebra and its Applications* 88–89 (1987) 67–82. doi:10.1016/0024-3795(87)90103-0.
- [22] A. Rahimi, B. Brecht, Random features for large-scale kernel machines, *Advances in Neural Information Processing Systems* (2007).
- [23] M. Gu, S. C. Eisenstat, Efficient algorithms for computing a strong rank-revealing qr factorization, *SIAM Journal on Scientific Computing* 17 (4) (1996) 848–869. doi:10.1137/0917055.
- [24] C. C. Paige, M. A. Saunders, LSQR: An Algorithm for Sparse Linear Equations and Sparse Least Squares, *ACM Transactions on Mathematical Software (TOMS)* 8 (1) (1982) 43–71. doi:10.1145/355984.355989.ASSET/CF3C8EE0-D698-4E67-BD8A-BF0B25681EEC/ASSETS/355984.355989.FP.PNG.
URL <https://dl.acm.org/doi/pdf/10.1145/355984.355989>
- [25] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, JAX: composable transformations of Python+NumPy programs (2018).
URL <http://github.com/google/jax>
- [26] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald,

- A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 Contributors, SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python, *Nature Methods* 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.
- [27] G. W. Stewart, The efficient generation of random orthogonal matrices with an application to condition estimators, *SIAM Journal on Numerical Analysis* 17 (3) (1980) 403–409. doi:10.1137/0717034.
 - [28] J. A. Tropp, A comparison principle for functions of a uniformly random subspace, *Probability Theory and Related Fields* 153 (3–4) (2011) 759–769. doi:10.1007/s00440-011-0360-9.
 - [29] P. Diaconis, M. Shahshahani, The subgroup algorithm for generating uniform random variables, *Probability in the Engineering and Informational Sciences* 1 (1) (1987) 15–32. doi:10.1017/S0269964800000255.
 - [30] M. W. Meckes, *The Random Matrix Theory of the Classical Compact Groups*, Vol. 218 of Cambridge Tracts in Mathematics, Cambridge University Press, 2019. doi:10.1017/9781108642324.
URL <https://doi.org/10.1017/9781108642324>
 - [31] D. Komatitsch, R. Martin, An unsplit convolutional perfectly matched layer improved at grazing incidence for the seismic wave equation, *Geophysics* 72 (5) (2007) SM155–SM167.